The Open Group Preliminary Standard

O-PAS™ Standard, Version 2.1

Part 6.1 – Information and Exchange Models: Overview and Interfaces

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Copyright © 2021, The Open Group

No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without the prior permission of the copyright owners.

Any use of this publication for commercial purposes is subject to the terms of the Annual Commercial License relating to it. For further information, see www.opengroup.org/legal/licensing.

The Open Group Preliminary Standard

O-PAS™ Standard, Version 2.1: Part 6.1 – Information and Exchange Models: Overview and Interfaces

Document Number: P210-6a

Published by The Open Group, May 2021.

Comments relating to the material contained in this document may be submitted to:

The Open Group, Apex Plaza, Forbury Road, Reading, Berkshire, RG1 1AX, United Kingdom

or by electronic mail to:

ogspecs@opengroup.org

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Contents

1 Introduction 1
- 1.1 Objective 1
- 1.2 Overview 1
- 1.3 Conformance 2
- 1.4 Normative References 2
- 1.5 Terminology 3
- 1.6 Future Directions (Informative) 4
  - 1.6.1 Import/Export 4
  - 1.6.2 Download/Upload 5
  - 1.6.3 Configuration Management 5
  - 1.6.4 Layer-F Application Management 5

2 Terms and Definitions 6
- 2.1 OPAF Glossary and Abbreviations 6

3 Information Model Structure (Informative) 7
- 3.1 General Information Models 7
- 3.2 Information Models 7

4 Cybersecurity 8

5 Import/Export Format 9
- 5.1 Overview 9
  - 5.1.1 Open Packaging Conventions 9
  - 5.1.2 AML 9
  - 5.1.3 Use of the AML in the O-PAS Standard 10
  - 5.1.4 Import/Export Package File 10
- 5.2 Conformance Requirements 12
  - 5.2.1 File Structure Requirements 12
  - 5.2.2 Conformant Engineering Tools and AML Attribute Types, Interface Classes, Roles Classes, and System Unit Classes 13
  - 5.2.3 AML 18
  - 5.2.4 Expressing Part Relationships of the Package in the AML 21

6 Download/Upload Exchange Models (Informative) 26

7 Layer-F Configuration Management Interface (Informative) 27
- 7.1 Overview 27
- 7.2 Download/Upload File Services 27
- 7.3 Switch Configuration Environments 28

8 Layer-F Application Management Interface (Informative) 29
- 8.1 Application Management States 29
- 8.2 ExecutionStateStateMachineType 31

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

8.3 ExecutionStateStateMachine Methods 35
8.3.1 ExecutionState Command Method Template 35
8.3.2 Start Method 36
8.3.3 Stop Method 36
8.3.4 QuickShutdown Method 36
8.3.5 QuickStartup Method 37
8.3.6 PlaceInMaintenance Method 37
8.3.7 PlaceOutOfService Method 37
8.3.8 ReturnToService Method 37
8.3.9 ReturnToOperating Method 38

9 O-PAS Profiles and Facets 39
9.1 Overview 39
9.2 Profiles and Facets 39
9.2.1 OBC-F-101: O-PAS Package File Facet 39

A O-PAS Part-Properties (Normative) 40
A.1 XML Schema for O-PAS Part-Properties 40

B UML (ISO/IEC 19501) Model Representations (Informative) 42

C Mapping UML to the AML (Informative) 43

D OPC UA (IEC 62541) Model Representations (Informative) 45

E Using AML Configuration Files to Specify OPC UA Nodes (Normative) 46
E.1 AML Definitions 47
E.2 Example Usage of UA-NodeAbsolute (Informative) 54
E.3 Example of UA-NodeRelative (Informative) 56

F Common Relationships between AML, OPC UA Nodeset, and Code Files in a .opas Package (Informative) 58
F.1 Overview of the Package Contents and Use 58
F.2 Interaction between the Files 59
F.3 Consuming the Package 60
F.4 AML Definitions 61
F.5 Example 61

G Formal AML Definitions (Normative) 66

H Abbreviations 67

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Preface

The Open Group

The Open Group is a global consortium that enables the achievement of business objectives through technology standards. Our diverse membership of more than 800 organizations includes customers, systems and solutions suppliers, tools vendors, integrators, academics, and consultants across multiple industries.

The mission of The Open Group is to drive the creation of Boundaryless Information Flow™ achieved by:

- Working with customers to capture, understand, and address current and emerging requirements, establish policies, and share best practices
- Working with suppliers, consortia, and standards bodies to develop consensus and facilitate interoperability, to evolve and integrate specifications and open source technologies
- Offering a comprehensive set of services to enhance the operational efficiency of consortia
- Developing and operating the industry's premier certification service and encouraging procurement of certified products

Further information on The Open Group is available at www.opengroup.org.

The Open Group publishes a wide range of technical documentation, most of which is focused on development of Standards and Guides, but which also includes white papers, technical studies, certification and testing documentation, and business titles. Full details and a catalog are available at www.opengroup.org/library.

This Document

This document is Part 6.1 of the O-PAS™ Standard, Version 2.1, a Preliminary Standard of The Open Group. It has been developed and approved by The Open Group.

The O-PAS Standard consists of the following 12 parts (of the anticipated 13 parts):

- O-PAS Part 1 – Technical Architecture Overview (Informative)
- O-PAS Part 2 – Security
- O-PAS Part 3 – Profiles (Informative)
- O-PAS Part 4 – O-PAS Connectivity Framework (OCF)
- O-PAS Part 5 – System Management
- O-PAS Part 6.1 – Information and Exchange Models: Overview and Interfaces (this document)

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

- O-PAS Part 6.2 – Information and Exchange Models: Basic Configuration
- O-PAS Part 6.3 – Information and Exchange Models: Alarm and Events Configuration
- O-PAS Part 6.4 – Information and Exchange Models: Function Blocks
- O-PAS Part 6.5 – Information and Exchange Models: IEC 61499
- O-PAS Part 6.6 – Information and Exchange Models: IEC 61131
- O-PAS Part 7 – Distributed Control Platform

The parts listed above are separate documents that can be updated and re-versioned as required as we move forward with the O-PAS Standard.

The O-PAS Standard, Version 2.1 is being published initially as a Preliminary Standard since it addresses an emerging area of technology; therefore, it may change before being published as a full standard of The Open Group. In such a case it will be made as upwards-compatible as possible with the corresponding Preliminary Standard, but complete upwards-compatibility is not guaranteed.

Once this document becomes a full standard, it will supersede the O-PAS Standard, Version 1.0 published in December 2019.

Conventions

A Glossary and Abbreviations reference is available. If a term is not defined in that document then the common English definition, as defined in Merriam-Webster's Collegiate Dictionary, applies.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Trademarks

ArchiMate, DirecNet, Making Standards Work, Open O logo, Open O and Check Certification logo, Platform 3.0, The Open Group, TOGAF, UNIX, UNIXWARE, and the Open Brand X logo are registered trademarks and Boundaryless Information Flow, Build with Integrity Buy with Confidence, Commercial Aviation Reference Architecture, Dependability Through Assuredness, Digital Practitioner Body of Knowledge, DPBoK, EMMM, FACE, the FACE logo, FHIM Profile Builder, the FHIM logo, FPB, Future Airborne Capability Environment, IT4IT, the IT4IT logo, O-AA, O-DEF, O-HERA, O-PAS, Open Agile Architecture, Open FAIR, Open Footprint, Open Process Automation, Open Subsurface Data Universe, Open Trusted Technology Provider, OSDU, Sensor Integration Simplified, SOSA, and the SOSA logo are trademarks of The Open Group.

Microsoft is a registered trademark of Microsoft Corporation in the United States and/or other countries.

UML is a registered trademark and Unified Modeling Language is a trademark of Object Management Group, Inc. in the United States and/or other countries.

All other brands, company, and product names are used for identification purposes only and may be trademarks that are the sole property of their respective owners.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Acknowledgements

The Open Group gratefully acknowledges the contribution of the following people in the development of this document:

- Dennis Brandl – BR&L Consulting, Inc.
- Alex Johnson – Schneider Electric
- Lars Ott – CODESYS Group

The Open Group gratefully acknowledges the contribution and leadership of the Co-Chairs of the Open Process Automation™ Forum and Working Groups.

Forum Co-Chairs

- Don Bartusiak – Collaborative Systems Integration, LLC
- Trevor Cusworth – Schneider Electric

Business Working Group Co-Chair

- Luis Duran – ABB Automation GmbH
- Jacco Opmeer – Shell

Enterprise Architecture Work Grouping Co-Chairs

- Dave Emerson – Yokogawa Electric Corporation
- Harirajan Padmanabhan – hpad, Inc.

Technology Working Group Co-Chairs

- David Fort – Rockwell Automation
- Kirk Smith – Intel Corporation

Certification Working Group Co-Chairs

- Ed Agis – Intel Corporation
- Ron Breault – Wind River Systems, Inc.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Referenced Documents

The following documents are referenced in Part 6.1 of the O-PAS Standard.

The documents are referred to in the text in such a way that some or all of their content constitutes requirements of this part. For dated references, only the version cited applies. For undated references, the latest version of the referenced document (including any amendments) applies.

(Please note that the links below are good at the time of writing but cannot be guaranteed for the future.)

Normative References

Normative references for Part 6.1 of the O-PAS Standard are defined in Section 1.4.

Informative References

- ISA 18: Instrument Signals and Alarms; refer to: https://www.isa.org/standards-and-publications/isa-standards/isa-standards-committees/isa18
- O-PAS Standard, Version 2.1: Part 1 – Technical Architecture Overview (Informative), The Open Group Preliminary Standard (P210-1), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210
- O-PAS Standard, Version 2.1: Part 6.2 – Information and Exchange Models: Basic Configuration, The Open Group Preliminary Standard (P210-6b), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210
- O-PAS Standard, Version 2.1: Part 6.3 – Information and Exchange Models: Alarm and Events Configuration, The Open Group Preliminary Standard (P210-6c), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

1 Introduction

1.1 Objective

The objective of Part 6 of the O-PAS Standard is to define the information and exchange models and methods that enable the portability of Layer-F applications among conformant engineering tools (see O-PAS Part 1 – Technical Architecture Overview (Informative)). Part 6.1 defines common services and the common information exchange structures used in the other subparts of Part 6 to manage Layer-F applications.

1.2 Overview

Part 6.1 defines:

- The standard format used for the exchange of import/export information across management applications
- The standard services used for the download/upload of information to O-PAS conformant components
- The standard services used to control Layer-F applications in O-PAS conformant components

The following namespace is used in this document:

NamespaceURI	Namespace Index	Example
http://opcfoundation.org/UA/	0	0:EngineeringUnits

Each of the other subparts of Part 6 is a separate document, identified as a number 6.x, where "x" is a number starting at 2. (Refer to the Preface for the subpart titles.) Each subpart defines a related set of information and exchange models, such as basic configuration, alarm models, or function block models.

Subparts of Part 6 may include the following elements as appropriate (some are not yet normative):

- A definition or reference to a UML® Class Model or other formal information model
- A definition or reference describing how that part uses the O-PAS import/export file format defined in Chapter 5 to exchange basic configuration information and Layer-F applications
- A definition or reference to the information and exchange model for the OCF (defined in O-PAS Part 4 – O-PAS Connectivity Framework (OCF)) that is used in the operation of the O-PAS conformant product

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

- Any O-PAS defined constraints on a referenced standard, or identification of any missing information or specification in a referenced standard, which would prevent or hinder portability of the Layer-F application

1.3 Conformance

Part 6.1 of the O-PAS Standard is normative, and conformance requirements (marked with "CRQ" identifiers) are defined later in this document.

1.4 Normative References

The following standards contain provisions which, through references in this document, constitute provisions of Part 6.1 of the O-PAS Standard. At the time of publication, the editions indicated were valid. All standards are subject to revision, and parties to agreements based on this document are encouraged to investigate the possibility of applying the most recent editions of the standards listed below.

- ANSI/ISA-62443-4-2:2018: Security for Industrial Automation and Control Systems – Part 4-2: Technical Security Requirements for IACS Components (adopted by IEC as IEC 62443-4-2)
- ECMA-376: Office Open XML File Formats, ECMA International; refer to: https://www.ecma-international.org/publications/standards/Ecma-376.htm
- Extensible Markup Language (XML) 1.0 (Third Edition), W3C Recommendation 04 February 2004, available at http://www.w3.org/TR/2004/REC-xml-20040204/
- IEC 61131-3:2013: Programmable Controllers – Part 3: Programming Languages; refer to: https://webstore.iec.ch/publication/4552
- IEC 61499: Function Blocks for Embedded and Distributed Control Systems Design, Third Edition, ISA
- IEC 62424:2016: Representation of Process Control Engineering – Requests in P&I Diagrams and Data Exchange between P&ID Tools and PCE-CAE Tools
- IETF RFC 5646: Tags for Identifying Languages, September 2009; refer to: https://tools.ietf.org/html/rfc5646
- IEC 62714-1:2018: Engineering Data Exchange Format for Use in Industrial Automation Systems Engineering – Automation Markup Language (AutomationML) – Part 1: Architecture and General Requirements; refer to: https://webstore.iec.ch/publication/32339
- IETF RFC 2119: Key Words for Use in RFCs to Indicate Requirement Levels, March 1997; refer to: https://tools.ietf.org/html/rfc2119
- ISO 3166: Country Codes; refer to: https://www.iso.org/iso-3166-country-codes.html
- ISO 639-1:2002: Codes for the Representation of Names of Languages – Part 1: Alpha-2 Code; refer to: https://www.iso.org/standard/22109.html

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

- ISO/IEC 19501: Information Technology – Open Distributed Processing – Unified Modeling Language™ (UML®), Version 1.4.2
- ISO/IEC 29500-2: Information Technology – Document Description and Processing Languages – Office Open XML File Formats – Part 2: Open Packaging Conventions
- O-PAS Standard, Version 2.1: Part 2 – Security, The Open Group Preliminary Standard (P210-2), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210
- O-PAS Standard, Version 2.1: Part 4 – O-PAS Connectivity Framework (OCF), The Open Group Preliminary Standard (P210-4), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210
- O-PAS Standard, Version 2.1: Part 6.1 – Information and Exchange Models: Overview and Interfaces, The Open Group Preliminary Standard (P210-6a), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210
- O-PAS Standard, Version 2.1: Part 6.5 – Information and Exchange Models: IEC 61499, The Open Group Preliminary Standard (P210-6e), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210
- O-PAS Standard, Version 2.1: Part 6.6 – Information and Exchange Models: IEC 61131, The Open Group Preliminary Standard (P210-6f), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210
- Unified Architecture (UA) Specification, Part 4: Services (also IEC 62541-4:2015), OPC Foundation; refer to: http://www.opcfoundation.org/UA/Part4/
- Unified Architecture (UA) Specification, Part 5: Information Model (also IEC 62541-5, OPC Foundation; refer to: http://www.opcfoundation.org/UA/Part5/
- Unified Architecture (UA) Specification, Part 6: Mappings (also IEC 62541-6, OPC Foundation; refer to: http://www.opcfoundation.org/UA/Part6/
- Unified Architecture (UA) Specification, Part 9: Alarms and Conditions (also IEC 62541-9:2015), OPC Foundation; refer to: http://www.opcfoundation.org/UA/Part9/
- Unified Architecture (UA) Specification, Part 13: Aggregates, OPC Foundation; refer to: http://www.opcfoundation.org/UA/Part13/
- Unified Architecture (UA) Specification, Part 17 (OPC 10000-17): Alias Names (also IEC 62541-17), OPC Foundation; refer to: http://www.opcfoundation.org/UA/Part17/
- Unified Architecture (UA) Specification, Part 200: Industrial Automation Model, OPC Foundation; refer to: https://opcfoundation.org/developer-tools/specifications-unified-architecture/part-200-industrial-automation-model/

1.5 Terminology

The key words “shall”, “shall not”, and “should” are used in the O-PAS Standard as defined below. These definitions are taken from IETF RFC 2119.

Shall Definition is an absolute requirement of the specification.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Shall not — Definition is an absolute prohibition of the specification.

Should — There may exist valid reasons in particular circumstances to ignore a particular item, but the full implications must be understood and carefully weighed before choosing a different course.

1.6 Future Directions (Informative)

1.6.1 Import/Export

1.6.1.1 Support for Document Management Systems

A future version of the O-PAS Standard will address additional functionality required for portability of Layer-F applications across different vendor systems and address security related to the interface. At that time, Part 6.1 and its contents will be revised and updated accordingly.

One such expected change is to support environments like the git source code management tool for storage of the artifacts.

Consider Figure 1 below:

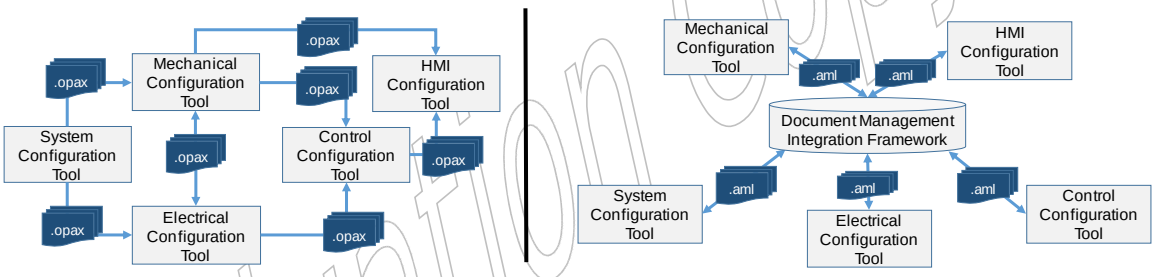


Figure 1: Direct Exchange of .opas Files Among Different Configuration Tools

On the left of Figure 1, each of the O-PAS configuration files is bundled and transferred as a package – i.e., an Open Process Exchange (.opas) file as defined in Chapter 5 – among the suite of engineering tools. The .opas file defined in this version is designed for this type of exchange.

However, it is recognized that storing content of the O-PAS configuration files in a document management system is valuable and that the current approach using .opas files is ill-suited for that purpose because .opas files are compressed and are not the single text file expected by many such tools; e.g., git. Later releases of the O-PAS Standard will define an alternate approach more suited to formal document management. We anticipate a solution that resembles the right-hand side of Figure 1.

In this scenario, the component files of the .opas file would be stored within the document management system along with an understanding of how they should be bundled on extraction. An Automation Markup Language (AutomationML or AML) file is used as an illustration.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

1.6.1.2 *Support for Module Type Packages (MTP)*

The .opas file concept was taken from the NAMUR MTP concept.[1] At the time the .opas format was defined, English translations and an official standard for MTP did not exist. It is expected that the O-PAS Standard will eventually allow import/export using the emerging MTP standard.

1.6.1.3 *Security Issues*

The import/export file needs security protection both when in the editing tool (in transit) and when written to its static form (at rest). The open item is how to implement those protections.

1.6.2 Download/Upload

Chapter 6 indicates our current plans concerning a standard approach to moving configuration and executable files into and out of a Distributed Control Node (DCN). It is presented to draw comments and suggestions.

It will include a description of the anticipated definition or reference to a download/upload model that may be used by conformant tools to download and upload the configuration information and Layer-F applications to an O-PAS conformant product.

When the definition or reference is not provided by a subpart of Part 6, the vendor-specific formats are used for downloading and uploading using the Configuration Management Interface methods.

As this section matures, it will incorporate specific instructions on security concerns.

1.6.3 Configuration Management

Chapter 7 will contain a description of a service that facilitates the download and upload of Layer-F applications. The current version shows the current raw ideas.

In particular, it will define a mechanism to run a newly installed or updated Layer-F application in an “open-loop” mode so that its behavior may be examined before it is fully deployed. It will also support roll-back of downloaded Layer-F applications.

As this section matures, it will incorporate specific instructions on security concerns around the myriad of attack points.

Clearly, this section will need to be reconciled with or moved to the proposed O-PAS Standard Part on Orchestration and Part 8 on Application Portability.

1.6.4 Layer-F Application Management

Chapter 8 indicates our current plans concerning a standard approach to control the operation of a Layer-F application running in a DCN. It is presented to draw comments and suggestions.

As this section matures, it will incorporate specific instructions on security concerns around the myriad of attack points.

Clearly, this section will need to be reconciled with or moved to the proposed O-PAS Standard Part on Orchestration and Part 8 on Application Portability.

[1] Refer to: https://www.namur.net/en/focus-topics/automation-modular-plants.html.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

2 Terms and Definitions

For the purposes of the O-PAS Standard, the following terms and definitions apply. Merriam-Webster's Collegiate Dictionary should be referenced for terms not defined in this chapter. The OPAF Glossary and Abbreviations should be referenced for additional terms not defined in this chapter.

2.1 OPAF Glossary and Abbreviations

For the OPAF Glossary and Abbreviations, see the definitions at: www.opengroup.org/open-process-automation-forum/glossary.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

3 Information Model Structure (Informative)

3.1 General Information Models

O-PAS conformant components used in a system have their interfaces defined through multiple information and exchange models. The associated information models are defined in the subparts of Part 6 and contain definitions or identification of projects, physical platforms, DCNs, signals, alarms and conditions, O-PAS function block applications, IEC 61131-3 applications, and IEC 61499 applications. A system will also contain components that are not defined in the O-PAS Standard, such as networking infrastructure devices, Human-Machine Interfaces (HMIs), and historians.

3.2 Information Models

The "Information Model" section within other subparts of Part 6 either defines or provides a reference to an implementation-independent information model. This information model defines how each subpart of Part 6 manages the structure and rules for information, without regard to the implementation technology.

The information model provides the rules and structure for the technology-specific information and exchange models (for example, AML and OPC UA), which have different ways of representing the same information.

If the O-PAS part bases its information model on an information model from some standard, that part of the O-PAS Standard references the existing standard's models and provide a link to the standard or to the organization that owns that standard and the document identifier within the organization; e.g., IEC 61499 or ISA 18.

If a suitable information model based on an existing standard is not found, the part of the O-PAS Standard does at least one of the following:

- Define an O-PAS specific information model
- Reference an informal information model
- Reference an implementation model (such as an OPC UA companion specification information model)

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

4 Cybersecurity

All O-PAS conformant facilities described in O-PAS Part 6.1 shall conform with ANSI/ISA-62443-4-2 SL2; refer to O-PAS Part 2 – Security for the applicable requirements; e.g., data at rest requirements would apply to an implementation of import/export [CRQ-P6.1-0070].

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

5 Import/Export Format

5.1 Overview

The import/export format section within other subparts of Part 6 either defines or provides a reference to an AML specification for exchanging configuration information and Layer-F applications. See Appendix C for general rules for mapping an information model based on a UML Class Model into an import/export format based on the AML.

The purpose of the import/export format is to define the configuration information and Layer-F applications that can be imported and exported among configuration tools from different vendors.

The import/export format uses a compressed Open Packaging Conventions (ISO/IEC 29500-2) package containing AML (IEC 62714-1) information describing the project. The AML is an XML-based format for the exchange of configuration information and Layer-F applications, following the CAEX (Computer Aided Engineering eXchange) format defined in IEC 62424.

5.1.1 Open Packaging Conventions

The Open Packaging Conventions (ISO/IEC 29500-2) is a standard for package files. Package files are containers holding a collection of parts and relationships between the parts. Information about the content types of parts (files contained in the package) and the relationships between parts are stored in standardized XML files. See ISO/IEC 29500-2 or ECMA-376, 4th Edition for more details.[2]

Open Packaging Conventions packages are usually stored as zip files.

5.1.2 AML

The AML is an international standard (IEC 62714-1) which allows for consistent data exchange among and across different configuration tool chains. Additional information is available at www.automationml.org.

The following description is taken from IEC 62714-1:2018:

"IEC 62714-1:2018 is a solution for data exchange focusing on the domain of automation engineering. The data exchange format defined in the IEC 62714 series (AML) is an XML schema-based data format and has been developed in order to support the data exchange in a heterogeneous engineering tools landscape. The goal of the AML is to interconnect engineering tools in their different disciplines; e.g., mechanical plant engineering, electrical design, process engineering, process control engineering, HMI development, PLC programming, robot programming, etc. The AML stores engineering information following an object-oriented paradigm and allows modeling of physical and logical plant components as data objects encapsulating different aspects. An object may consist of other sub-objects and may itself be part

[2] See also https://en.wikipedia.org/wiki/Open_Packaging_Conventions for a less formal description.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

of a larger composition or aggregation. Typical objects in plant automation comprise information topology, geometry, kinematics, and logic, whereas logic comprises sequencing, behavior, and control."

The AML allows the combination of existing industry data formats used for the storage and exchange of different aspects of engineering information into one consistent format. These non-AML data formats are used "as-is" within their own specifications and are not branched for AML needs. The core of the AML is the top-level data format CAEX that connects the different data formats.

5.1.3 Use of the AML in the O-PAS Standard

The O-PAS Standard uses the AML for two purposes:

1. To provide information to control system editors of various types needed to specify the content of the O-PAS import/export files; e.g., to specify which parameters are optional and the data types of those parameters.
2. To provide a file format to hold the configured information for exchange among tools used with O-PAS designed equipment.

To achieve these goals a number of conventions are defined below.

5.1.4 Import/Export Package File

The import/export package file is identified with the extension ".opas", and is a zipped (compressed) file identified as a package in ISO/IEC 29500-2.

For example, a package holding an AML document with a source file and an OPC UA nodeset file contains five parts: the AML XML of the document, the Core Properties part, the O-PAS Part-Properties part, and two other parts representing the source file and the nodeset file.

A package provides a convenient way to distribute an O-PAS import/export file with all constituent parts, such as an XML file, download files, and source files. It also provides a method to allow digitally signed and/or encrypted documents within the package which have links to other documents in the package. (Note that in this version of the standard there are no profiles that contain signing and encrypting parts.)

Parts often have references to other parts.

In the example above, we first have a relationship from the package to the central AML file part, the Core Properties part, and the O-PAS Parts-Properties part. The AML file then has relationships to the source file and the nodeset file. The source file also has a relationship to the nodeset file.

Relationships can point to other parts within the package or be external; that means they point to a URI outside of the package. A good example is a standardized nodeset file that is hosted on a server of the standards organization defining it. This file can be easily accessed using its URI and thus does not have to be present in the package.

References from within the files contained in an .opas package make use of the existing relationships within the package. For example, if the AML file wants to reference a code file, then there must be a part relationship in the package from the AML file part to the part for that

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

code file. Inside the AML file, the relationship ID for that reference is then used to point to the code file. See Section 5.2.4 for details.

The structure of files in the package is of minor importance since relationships are used to build logical connections between the different parts in the package.

A .opas package is expected to contain a minimum set of parts. They are listed with the relationship types for their package relationships in Table 1.

Table 1: Minimum Parts in a .opas Package and their Package Relationships

Part	Relationship Type	More Information
Root AutomationML	http://schemas.automationml.org/container/relationship/RootDocument	IEC 62714-1
Core Properties	http://schemas.openxmlformats.org/package/2006/relationships/metadata/core-properties	ISO/IEC 29500-2
O-PAS Part-Properties	http://opengroup.org/open-process-automation/O-PAS/relationships/part-properties-target	Appendix A

Notes:

1. Relationship type is defined in ISO/IEC 29500-2 and ECMA-376.
2. The Core Properties relationship is defined in the AML White Paper.[3]
3. The relationship URL/URI for O-PAS Part-Properties is valid down to relationships. The O-PAS Standard is adding the rest.

Core Properties provides metadata on the package as a whole and O-PAS Part-Properties provides metadata for individual parts of the package.

A .opas package may contain parts that contain AML. The AML in these parts may reference additional files. Those additional files may be included in the package or they may be external to the package. To allow for these files to be found, the Open Packaging Conventions standard specifies how to build a relationship between the reference in an AML attribute and the additional file. This relationship is itself stored in a file in the .opas package. This standardized relationship allows tools using the .opas file to find the additional files when required.

Other subparts of Part 6 – for example, Part 6.5 (concerning IEC 61499 implementations) and Part 6.6 (concerning IEC 61131-3 implementations) – leverage this capability to reference and include their required files. The referenced files, the AML interface class for the referenced files, the content type for the referenced files, and the relationship type for that type of relationship are specified as required by the other subparts of Part 6.

The InstanceHierarchies in the root AML serve for the storage of individual and project-related engineering information. They form the center of the ".aml" top-level format and contain all individual data objects including properties, interfaces, relations, and links. They are defined using configuration management tools that follow the structure defined for a specific project.

[3] Refer to https://www.automationml.org/o.red.c/publications.html.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

The following provisions apply to the import/export format (see IEC 62714-1, §7.7 for AML provisions):

1. The import/export format does not restrict the depth of an InstanceHierarchy level.
2. While the AML standard does not restrict the architecture rules of an InstanceHierarchy, this document and other subparts of Part 6 do. The O-PAS Standard provides rules that allow all tools to understand the architecture of the InstanceHierarchy.
3. The import/export format does not define naming conventions for an InstanceHierarchy.
4. Every object within an InstanceHierarchy is assigned to a role class in order to identify its abstract type.

5.2 Conformance Requirements

This section formally lists the requirements for an O-PAS import/export exchange file.

5.2.1 File Structure Requirements

The following requirements apply to any tool used to manipulate the contents of a .opas file:

1. A conformant configuration tool shall use a zipped file format as defined in ISO/IEC 29500-2, with the extension ".opas", which contains a package relationships item and a content types item. [CRQ-P6.1-0000]
2. A conformant configuration tool shall use the Content Type shown in Table 2 for any part in the package that is a File Type shown in Table 2. [CRQ-P6.1-0001]
3. A conformant configuration tool shall include an AML (IEC 62714-1) file with the extension .aml in the ".opas" file, following the format defined in IEC 62424 CAEX. For the part containing the file, there shall be a package relationship from the package to the part using the relationship type "http://schemas.automationml.org/container/relationship/RootDocument". [CRQ-P6.1-0002]
4. A conformant configuration tool shall include in the package an O-PAS Part-Properties part as defined in Appendix A. For this part, there shall be a package relationship from the package to the O-PAS Part-Properties part using the relationship type "https://opengroup.org/open-process-automation/O-PAS/relationships/part-properties". [CRQ-P6.1-0003]

   There shall also be a part relationship from the O-PAS Part-Properties part to all parts for which metadata properties are included. The relationship type shall be "http://opengroup.org/open-process-automation/O-PAS/relationships/part-properties-target". [CRQ-P6.1-0004]
5. A conformant configuration tool shall include in the package a Core Properties part as defined in ISO/IEC 29500-2. [CRQ-P6.1-0005]
6. A conformant configuration tool shall support the use of Basic-OpenPackagingConventionRelationship in AML files instead of ExternalDataReference to link to other parts/files of the package (internal or external). [CRQ-P6.1-0006]

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

7. Support for the use of OpenPackagingConventionRelationship in AML files requires that there shall be a part relationship from the AML part to the part to be referenced. [CRQ-P6.1-0007]

8. A conformant configuration tool shall retain all non-modified information that was included in an imported file into an exported file. [CRQ-P6.1-0008]

   Note: This basically means that a tool does not delete or change information in the configuration file which it does not directly modify.

9. A conformant configuration tool shall not modify the AutomationMLBaseRoleClassLib library. [CRQ-P6.1-0009]

10. A conformant configuration tool shall not modify the AutomationMLInterfaceClassLib library. [CRQ-P6.1-0010]

11. A conformant configuration tool shall not modify the AutomationMLAttributeTypeLib library. [CRQ-P6.1-0011]

Table 2: Content Types in a .opas Package (Non-Exclusive)

File Type	Content Type	Description
AutomationML	model/vnd.automationml+xml	An AML file as defined in IEC 62714-1 and AML §8.9.
O-PAS Part-Properties xml	application/vnd.opas.partproperties+xml	See Appendix A.
Core Properties xml	application/vnd.openxmlformats-package.core-properties+xml	See ISO/IEC 29500-2 and ECMA-376.
OPC UA Nodeset2 xml	application/vnd.opc.ua.nodeset2+xml	See OPC UA Part 6.

Note: The Content is a MIME mapping.

5.2.2 Conformant Engineering Tools and AML Attribute Types, Interface Classes, Roles Classes, and System Unit Classes

5.2.2.1 Conformant Engineering Tools and the AML

A conformant engineering tool set shall define AML attribute types per the bulleted items in Part 6.1 Section 5.2.2.2. [CRQ-P6.1-0012]

A conformant engineering tool set shall define AML interface classes per the bulleted items in Part 6.1 Section 5.2.2.3. [CRQ-P6.1-0013]

A conformant engineering tool set shall define AML role classes per the bulleted items in Part 6.1 Section 5.2.2.4. [CRQ-P6.1-0014]

A conformant engineering tool set shall define AML System Unit Classes (SUCs) per the bulleted items in Part 6.1 Section 5.2.2.5. [CRQ-P6.1-0015]

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

5.2.2.2 O-PAS Requirements on AML Attribute Type Definitions

Attribute types model user-specific, company-specific, region-specific, country-specific, or international standard attribute semantics.

The following requirements apply to any conformant engineering tool that manipulates a .opas file:

1. An attribute type defined by a conformant engineering tool shall include an attribute type ID in accordance with the AML editor's usage and IEC 62714-1. [CRQ-P6.1-0016]
    a. The ID used to identify this specific attribute type. If needed, the attribute type is defined in a subpart of Part 6 and is the normative definition of the attribute type. This is included in the .aml file associated with the subpart. Once created, the ID does not change over the lifetime of the corresponding object within all participating tools. This information is defined in the attribute type element's ID tag. In order to support portability of configurations, and provide quick identification of O-PAS attribute types, the attribute type ID shall be a GUID. [CRQ-P6.1-0063]
2. An attribute type defined by a conformant engineering tool shall include a name in accordance with the AML editor's usage and IEC 62714-1. [CRQ-P6.1-0017]
    a. The name of the attribute type is as seen in the AML editor. This is defined in a subpart of Part 6 and is the normative definition of the attribute type. This shall be included in the .aml file associated with the subpart and defined in the attribute type element's name tag. [CRQ-P6.1-0064]
3. An attribute type defined by a conformant engineering tool shall include a description in accordance with the AML editor's usage and IEC 62714-1. [CRQ-P6.1-0018]
    a. The attribute type "Description" is a longer description of the attribute type, referencing any other O-PAS parts or external standards as needed to describe the meaning of the attribute type. The attribute type description shall assist in understanding the attribute type and shall be included in the .aml file associated with the subpart. [CRQ-P6.1-0065]
4. An attribute type defined by a conformant engineering tool may include one or more relations in accordance with the AML editor's usage and IEC 62714-1. [CRQ-P6.1-0019]
    a. Relations are optional and contain the full path from the class to its base class.
5. An attribute type defined by a conformant engineering tool shall include a data type for the attribute in accordance with the AML editor's usage and IEC 62714-1. [CRQ-P6.1-0020]
    a. The data types are defined using the native data types of XML (xs:int, xs:Boolean, xs:token, etc.).
6. An attribute type defined by a conformant engineering tool may include a default value for the attribute in accordance with the AML editor's usage and IEC 62714-1. [CRQ-P6.1-0021]

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

7. An attribute type defined by a conformant engineering tool may include unit of measure for the attribute in accordance with the AML editor's usage and IEC 62714-1.

8. An attribute type defined by a conformant engineering tool may include constraints for the attribute in accordance with the AML editor's usage and IEC 62714-1. [CRQ-P6.1-0023]

    a. Constraints might include a set of predetermined allowed values or ordinal scaled values.

5.2.2.3 O-PAS Requirements on AML Interface Class Definitions

Interface classes are used to specify linkages. The linkages types are:

- Those between an AML internal element and a different AML element in the AML file's InstanceHierarchy
- Those between an internal element and an external element

The AML requires one interface class for each type of interface between elements. For example, there may be an interface class which represents the data output from a function block, and another interface which represents the data input to a function block. These interfaces are then linked together in the InstanceHierarchy to represent data sent from one function block to another function block.

The following requirements apply to a .opas file:

1. An interface class defined by a conformant engineering tool shall include an interface class ID in accordance with the AML editor's usage and IEC 62714-1. [CRQ-P6.1-0024]

    a. The ID used to identify this specific interface class. This is defined in a subpart of Part 6 and is the normative definition of the interface class. This is included in the .aml file associated with the subpart. Once created, the ID does not change over the lifetime of the corresponding object within all participating tools. This information is defined in the interface class element's ID tag. In order to support portability of configurations and provide quick identification of O-PAS interface classes, the interface class ID shall be a GUID defined in a subpart of Part 6. [CRQ-P6.1-0062]

2. An interface class defined by a conformant engineering tool shall include a name in accordance with the AML editor's usage and IEC 62714-1. [CRQ-P6.1-0025]

    a. The name of the interface class as seen in the AML editor. Names are defined in a subpart of Part 6 and is the normative definition of the interface class. This is included in the .aml file associated with the subpart. This information is defined in the interface class element's name tag. [CRQ-P6.1-0049]

3. An interface class defined by a conformant engineering tool shall include a description in accordance with the AML editor's usage and IEC 62714-1. [CRQ-P6.1-0026]

    a. The description field contains a full description of the interface's purpose. The description may reference any other O-PAS parts or external standards as needed to describe the meaning of the interface class. The description should assist in understanding the interface. [CRQ-P6.1-0069]

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

4. An interface class defined by a conformant engineering tool may include zero or more relations in accordance with the AML editor's usage and IEC 62714-1. [CRQ-P6.1-0027]

    a. A relation is an optional characteristic of the interface class that contains the full path from the class to its base class.

In order to support version management of the interface class libraries, changes of a source class will lead to a new version of the class with another name. Within the new class, the full path of the old version of the class will be stored in the tag OldVersion. Additionally, within the old class, the path to the new version will be stored in the tag NewVersion (see IEC 62714-1, §5.5.2).

5.2.2.4 O-PAS Requirements on AML Role Class Definitions

Role classes are used to provide "semantic" information about objects. For example, there could be a role class called "Work Center", which implies certain functionality provided by an instance. The same role class may be referenced by many SUCs.

For example, there may be multiple SUCs for different function blocks, but each of them could implement the same role class used to represent an input parameter. Another example would be use of role classes to represent the event, condition, and alarm types defined by the OPC UA.

Within an AML file, a role class is associated with an SUC by means of the SUC supported role class elements. The association of a role class with an SUC indicates that the class is able to support the referenced role(s).

Multiple role classes can be associated with a single SUC. Once an SUC is instantiated in the InstanceHierarchy, the internal elements of the SUC hold instance information for any role class attributes and interfaces.

The role class "Group" is a role type for objects that serve for the grouping of objects (instances) that belong together from a certain engineering perspective. AML group objects reference the path AutomationMLBaseRoleClassLib/AutomationMLBaseRole/Group. The objects so grouped are known as "mirror objects".

In the AML, one object (instance), the "master object", may have a relation to another AML object (instance), the "mirror object". The mirror object is considered to be identical to the master object. This enables placing one object instance into different plant hierarchies and thus allows modeling of complex object networks with crossed structures.

The following requirements apply to a .opas file:

1. A role class defined by a conformant engineering tool shall have an ID in accordance with the AML editor's usage and IEC 62714-1. [CRQ-P6.1-0028]

    a. The ID used identifies this specific role class. If needed, the role class is defined in a subpart of Part 6 and is the normative definition of the role class. The role class ID is included in the .aml file associated with the subpart. Once created, the ID does not change over the lifetime of the corresponding object within all participating tools. The role class ID is defined in the role class element's ID tag. In order to support portability of configurations and provide quick identification of O-PAS role classes, the role class ID is a GUID defined in a subpart of Part 6. [CRQ-P6.1-0059]

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

2. A role class defined by a conformant engineering tool shall include a name in accordance with the AML editor's usage and IEC 62714-1. [CRQ-P6.1-0029]

    a. The name of the role class as seen in the AML editor. Names are defined in a subpart of Part 6 and are the normative definition of the role class. Role class names shall be included in the .aml file associated with the subpart and shall be defined in the role class element's name tag. [CRQ-P6.1-0030]

3. A role class defined by a conformant engineering tool shall include a description in accordance with the AML editor's usage and IEC 62714-1. [CRQ-P6.1-0031]

    a. The description field contains a full description of the interface's purpose. The description may reference any other O-PAS parts or external standards as needed to describe the meaning of the role class. The role class description shall assist in understanding the interface and be included in the .aml file. [CRQ-P6.1-0032]

4. A role class defined by a conformant engineering tool may include zero or more relations in accordance with the AML editor's usage and IEC 62714-1. [CRQ-P6.1-0033]

    a. A relation is an optional characteristic of the role class that contains the full path from the class to its base class.

5. A role class defined by a conformant engineering tool may include additional attributes that are associated with the role. These attributes are inherited by an SUC that implements the role. [CRQ-P6.1-0034]

6. In order to support version management of the role class libraries, a conformant engineering tool which makes changes to a source class shall create a new version of the class with another name. [CRQ-P6.1-0035]

7. In order to support version management of the role class libraries, a conformant engineering tool which makes changes to a source class shall store the full path of the old version of the class in the new class using the tag OldVersion. [CRQ-P6.1-0036]

8. In order to support version management of the role class libraries, a conformant engineering tool which makes changes to a source class shall store, within the old class, the path to the new version using the tag NewVersion (see IEC 62714-1, §5.5.2). [CRQ-P6.1-0037]

5.2.2.5 O-PAS Requirements on SUC Definitions

SUCs are used as templates to define instances in the InstanceHierarchy. Each SUC represents a class of exchanged objects. Each SUC shall have an assigned role class to allow for interpretability of the derived instances.

The following requirements apply to a .opas file:

1. An SUC defined by a conformant engineering tool shall have an ID in accordance with the AML editor's usage and IEC 62714-1. [CRQ-P6.1-0038]

    a. The ID is used to identify this specific SUC. If needed, the SUC is defined in a subpart of Part 6 and is the normative definition of the SUC. The SUC ID is included in the .aml file associated with the subpart. Once created, the ID does not change over the lifetime of the corresponding object within all participating tools.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

This information is defined in the SUC element's ID tag. In order to support portability of configurations and provide quick identification of O-PAS SUCs, the SUC ID is a GUID defined in a subpart of Part 6. [CRQ-P6.1-0060]

2. An SUC defined by a conformant engineering tool shall include a name in accordance with the AML editor's usage and IEC 62714-1. [CRQ-P6.1-0039]
    a. The name of the SUC as seen in the AML editor. Names are defined in a subpart of Part 6 and are the normative definition of the SUC. SUC names shall be included in the .aml file associated with the subpart and shall be defined in the role class element's name tag. [CRQ-P6.1-0061]
3. An SUC defined by a conformant engineering tool shall include a description in accordance with the AML editor's usage and IEC 62714-1. [CRQ-P6.1-0040]
    a. The description field contains a full description of the interface's purpose. The description may reference any other O-PAS parts or external standards as needed to describe the meaning of the SUC. The SUC description should assist in understanding the interface and be included in the .aml file. [CRQ-P6.1-0072]
4. An SUC defined by a conformant engineering tool may include zero or more relations in accordance with the AML editor's usage and IEC 62714-1. [CRQ-P6.1-0041]
    a. A relation is an optional characteristic of the SUC that contains the full path from the class to its base class.
5. In order to support version management of the SUC libraries, a conformant engineering tool which makes changes to a source class shall store the full path of the old version of the class in the new class using the tag OldVersion. [CRQ-P6.1-0042]
6. In order to support version management of the SUC libraries, a conformant engineering tool which makes changes to a source class shall store, within the old class, the path to the new version using the tag NewVersion (see IEC 62714-1, §5.5.2). [CRQ-P6.1-0066]

5.2.3 AML

5.2.3.1 Library Naming Conventions

The AML requires that users create libraries of attribute types, interface types, role classes, and SUCs.

A conformant engineering tool shall restrict the naming of the attribute types, interface types, role classes, and SUCs that it creates to the O-PAS naming conventions documented in O-PAS Part 6.1, Table 3 (AML Library Naming Conventions). [CRQ-P6.1-0043]

Table 3: AML Library Naming Conventions

Library	Naming Convention	Example Name	Comments on Example
Attribute Type	O-PAS-<AreaName>-AttributeTypeLib	O-PAS-General-AttributeTypeLib	This library holds all types expected to be used in multiple domains; e.g., MandatoryAttribute and OptionalAttribute.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Library	Naming Convention	Example Name	Comments on Example
Interface Type	O-PAS-<AreaName>-InterfaceTypeLib	O-PAS-Alarm-InterfaceTypeLib	This library holds all interfaces defined for use with alarms and conditions as indicated by the name.
Role Class	O-PAS-<AreaName>-RoleClassLib	O-PAS-Alarm-RoleClassLib	This library holds all role classes defined for use with alarms and conditions as indicated by the name.
System Unit Class	O-PAS-<AreaName>-System Unit ClassLib	O-PAS-Alarm-System Unit ClassLib	This library holds all SUCs for alarms and conditions as indicated by the name.

5.2.3.2 *Library Item Naming Conventions*

A conformant engineering tool shall generate restrict the naming of the libraries it creates to the O-PAS naming convention documented in O-PAS Part 6.1, Table 4 (AML Library Item Naming Conventions). [CRQ-P6.1-0044]

Table 4: AML Library Item Naming Conventions

Library	Naming Convention	Example Name	Comments on Example
Attribute	<UsageSpecificName>	ConditionalName OutOfServiceState	Attributes are the bottom of the AML hierarchy. Their pathname includes the owning library so their names do not require additional qualification to assure the attribute is uniquely identified. This specific attribute example is intended to indicate to the editor using the O-PAS AML whether or not the attribute must be included.
Interface	<AreaName>-<ReceivingAttributeName>	Alarm-InputNode	The interface name comes from the “sink” side of the link; that is, it explains how the connection will be used.
Role Class	<AreaName>-<UsageSpecficName>	Alarm-CommonInformation	In this example, the role class holds the attributes that are shared by all alarms and conditions. The suffix “Information” can be added because role classes carry information about SUCs created. Role classes are not used directly to create instances.
System Unit Class	<AreaName>-<UsageSpecificName>	Alarm-ExclusiveLevelType	The suffix “Type” can be added because SUCs are types that are used to create instances.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

5.2.3.3 *Naming Connectable Interface Types*

A conformant engineering tool shall create all connectable parameters in AML objects as AML interfaces. [CRQ-P6.1-0067]

A conformant engineering tool shall ensure that all created connectable parameters are named after the attribute that is the sink of the connection. [CRQ-P6.1-0068]

5.2.3.4 *Mandatory and Optional Attributes*

In defining configuration information for the OPC UA server required by the OCF, it is often required to indicate if a configuration parameter is optional or mandatory.

Therefore:

- A conformant engineering tool shall define a semantic element in the relations (RefSemantic) of "OPC UA Optional" for optional attributes [CRQ-P6.1-0045]
- Attributes without the RefSemantic of "OPC UA Optional" shall be considered mandatory for the OPC UA server by all applications that use the AML file [CRQ-P6.1-0046]

This convention is illustrated in Figure 2.

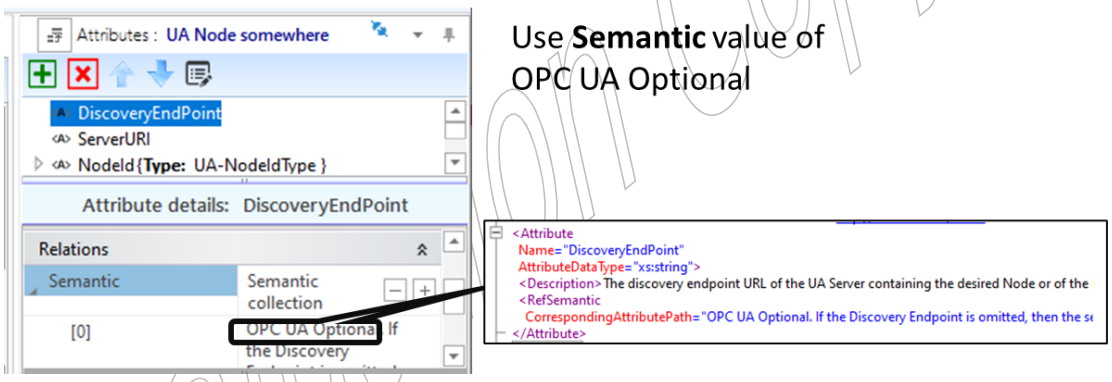


Figure 2: Optional Attribute Identification

5.2.3.5 *Localized Text Attributes*

Localized text is used to indicate text that should have a local text equivalent. This is used primarily in mapping of OPC UA localized text types, but may also be used for any text that could be displayed during design or runtime.

Modeling localized language texts requires nested attributes. For this purpose, the attribute with text to be localized contains sub-attributes.

Specifically, the base attribute is of type "xs:string". Its value will be the value used if the desired language/country pair is not found in the sub-attributes or the tool does not support localized text.

Sub-attributes are added to specify the language/country alternatives. These sub-attributes are named using the language and country pairs found in IETF RFC 5646. Examples of sub-attribute names include: "en-US", "de-DE", and "fr-FR". The five-character string uses the two-character

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

code for language from ISO 639-1, a hyphen, and the two-character code from ISO 3166 alpha-2 for country. The value of the attribute would be the actual alternative text.

Thus, these requirement statements define how to implement localized text in an O-PAS AML file:

- To indicate that an attribute supports localized text, a conformant engineering tool shall create a string (xs:string) attribute of whatever name, assign a default value for it, and specify a semantic element in the relations (RefSemantic) of "OPC UA Localizable Text" [CRQ-P6.1-0047]

  This last step allows for quick identification of text strings that should be localized.

- To specify the alternate text values for the attribute, a conformant engineering tool shall define one or more sub-attributes of the type: AutomationMLBaseAttributeTypeLib/LocalizedAttribute, name these sub-attributes using the language code for the target text according to IETF RFC 5646, and place the alternative text in the attributes value field [CRQ-P6.1-0048]

This is illustrated in Figure 3.

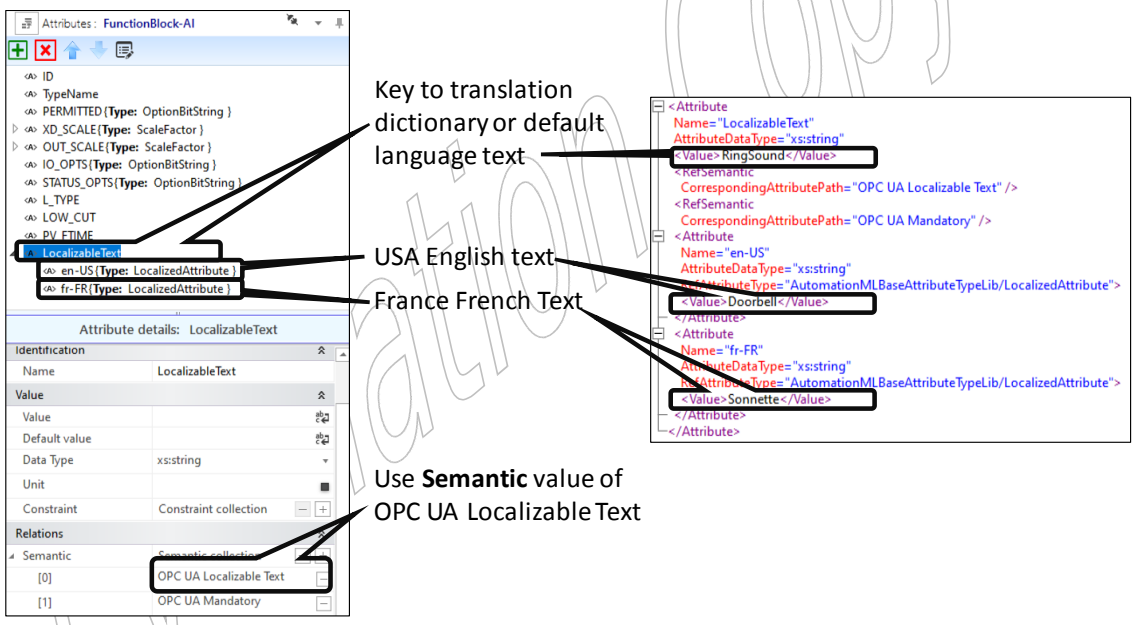


Figure 3: Localized Text Representation

5.2.4 Expressing Part Relationships of the Package in the AML

The Open Packaging Conventions standard (ISO/IEC 29500-2) defines a mechanism to store relationships from one part to another. Each of the relations has a unique ID in the .rels file of the part in which the relation originates. Relationships can also be "external" and point outside of the package; for example, to an XML schema file on a server. For more information, see ISO/IEC 29500-2 §9.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

The rationale behind this technology choice is as follows:

- These packages are valuable because:
  - — Libraries exist for common operating systems that handle reading and writing such packages; some examples can be found in:
    - — Microsoft® documentation: https://docs.microsoft.com/en-us/dotnet/api/system.io.packaging?view=dotnet-plat-ext-3.1
    - — Libopc: an open source platform-independent implementation
  - — Signing of individual parts and/or relationships of a part is fully supported; some examples can be found in:
    - — Microsoft documentation: https://docs.microsoft.com/en-us/dotnet/api/system.io.packaging.packagedigitalsignaturemanager?view=netcore-3.1
    - — https://github.com/qmuntal/opc
- The packages should improve the security and ease of implementation since they allow linking to other files from the AML base file

The following text shows the contents of the relations required to support the AML file in our .opas package. There are three relationships defined:

1. The AML wants to reference a file called “PartProperties.xml”. Since the PartProperites.xml file could be anywhere, the relationship record provides the pathname within the .opas file to the needed material. The “Type” specification …
2. The AML wants to a reference to an OPC UA nodeset that is outside the .opas file. The relationship allows the tool reading the AML to find it.
3. The AML contains a reference to a set of blocks known as “reallyAdvanced.blocks” and the relationship provides the pathname to those blocks in the .opas file.

Note: The following example refers to Root.aml as the base AML file for a project. The reader should understand that this is a sample name and is not a mandatory name for the “top-level” file.

The Root.aml.rels file for a Root.aml part in a .opas package could look like this:

```
<Relationships
xmlns="http://schemas.openxmlformats.org/package/2006/relationships">
  <Relationship
    Id="rId3"
    Type="http://opengroup.org/open-process-automation/O-
PAS/relationships/part-properties-target"
    Target="PartProperties.xml"/>
  <Relationship
    Id="rId2"
    Type=" application/vnd.opc.ua.nodeset2+xml "
    TargetMode=”External”
    Target="https://raw.githubusercontent.com/OPCFoundation/UA-
Nodeset/master/Schema/Opc.Ua.NodeSet2.xml"/>
  <Relationship
    Id="rId1"
    Type="http://blockbuilder.edu/blocks"
```


© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

```
    Target="reallyAdvanced.blocks"/>
</Relationships>
```

The AML and other files that are part of a .opas package shall utilize this relationship mechanism to link to other files that are parts in the package. The required interface classes and attributes are described in Section 5.2.4.x. [CRQ-P6.1-0050]

Figure 4 provides an example of an AML file using the Open Packaging Conventions relationships (through the relationship ID) to target an external file from within the AML. In this figure, InternalElement is part of the InstanceHierarchy and is used simply as an anchor for the reference mechanism being described. The FileReference attribute could be attached to any AML object and would typically be on a role class.

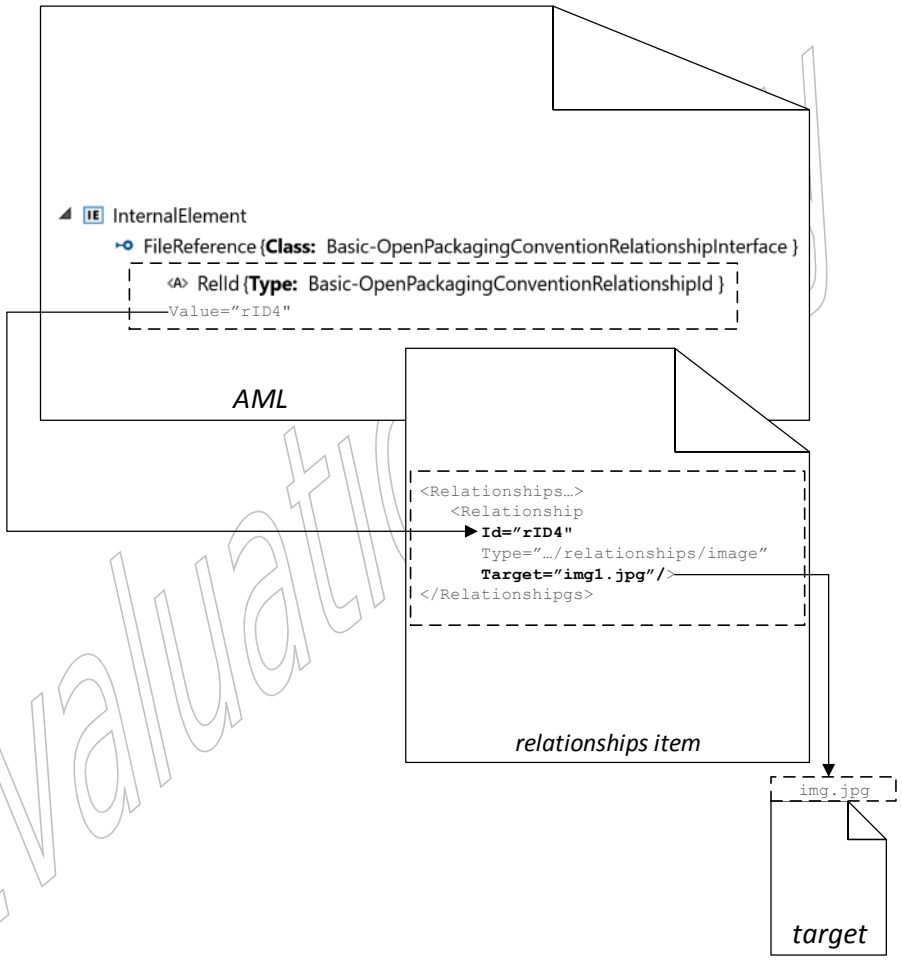


Figure 4: Example of Using the Open Packaging Conventions Relationships from within an AML File

5.2.4.1 *Basic-OpenPackagingConventionRelationshipId Attribute Type*

This attribute type is part of the O-PAS-Basic-AttributeTypeLib. The values for this attribute type are the same as in the "Id" XML-Attribute of the relationship element in a .rels file. Its data type is thus xs:ID with the same restrictions as imposed on "Id" in ISO/IEC 29500-2 §9.3.2.2.

Using the example above, three attributes of type Basic-OpenPackagingConventionRelationshipId exist and their values are "rId1", "rId2", or "rId3".

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Attributes of the type Basic-OpenPackagingConventionRelationshipId in the AML shall be used to hold the relationship ID used in the corresponding relationship item in the .rels file. [CRQ-P6.1-0051]

Figure 4 illustrates an example that is typical of associating an image file to an object in the InstanceHierarchy that needs a picture to represent it; e.g., a function block in an editor.

5.2.4.2 *Basic-OpenPackagingConventionRelationshipInterface Interface Class*

This interface class is part of the O-PAS-Basic-InterfaceClassLib and shall be used to link to other files (parts) internal or external to the package. Its attributes are formally defined in Table 5. It uses the OpenPackagingConventionRelationshipId attribute type defined in Section 5.2.4.1 and the intention is that more interface classes inheriting from this one are being created. These derived interface classes can then provide deep link capabilities into the target file; for example, an interface class targeting OPC UA nodeset files can be created that has additional attributes that allow targeting a specific UA node within that nodeset file.

A couple of these derived interface classes are defined in this document.

Table 5: Basic-OpenPackagingConventionRelationshipInterface Interface Class Definition

Name	Attribute Type	M/O/E	Comment
RelId	Basic-OpenPackagingConventionRelationshipId	M	This attribute is used to reference a relationship ID for one of the relationships in the .rels file of this AML file. This is used to create links to other parts of the package. Relationships may be external and point out of the package; e.g., to the OPC Foundation website where standard nodeset XML files are hosted. See ISO/IEC 29500-2 §9.

5.2.4.3 *Basic-CodeFileRelationInterface Class*

This interface class derives from Basic-OpenPackagingConventionRelationshipInterface and is used to link to files containing source code. Its attributes are formally defined in Table 6.

Table 6: Basic-CodeFileRelationInterface Interface Class Definition

Name	Attribute Type	M/O/E	Comment
RelId	Basic-OpenPackagingConventionRelationshipId	M	See Section 5.2.4.2. Target relationship must be to a part with a content type fitting for source code; e.g., application/java-script or text/x-java-source. Subparts of Part 6 can define suitable content types for their respective code files.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

5.2.4.4 *Basic-DownloadFileInterface Class*

This interface class derives from Basic-OpenPackagingConventionRelationshipInterface and is used to link to files containing download information as discussed in Chapter 6. Its attributes are formally defined in Table 7.

Table 7: Basic-DownloadFileInterface Interface Class Definition

Name	Attribute Type	M/O/E	Comment
RelId	Basic-OpenPackagingConventionRelationshipId	M	See Section 5.2.4.2. Target relationship must be to a part with a content type fitting for download files. Subparts of Part 6 can define suitable content types for their respective download files.

5.2.4.5 *Other Derived Interface Classes*

Other subparts of Part 6 can derive from the interface class Basic-OpenPackagingConventionRelationshipInterface or one of the derivates listed in this document to create interface classes specific for their files.

Examples for this can be found in Appendix E and Appendix F which detail how the OPC UA is integrated into .opas files.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

6 Download/Upload Exchange Models (Informative)

For the O-PAS Standard, the download and upload exchange file formats are defined by the vendors' configuration editing or configuration management tools to match the target DCN software.

Conformant engineering tools store their vendor-specific download/upload formatted files in the .opas file according to Section 5.2.4.4.

The approach allows tools to extract the download information from the .opas file. The download files may then be downloaded using methods that are not yet defined. The download/upload methods do not depend on the format of the information in the files.

There may be one or more download files generated for each DCN. For example, there may be an OPC UA nodeset file, a file containing a function block library in binary format, and a file containing a function block application for a single DCN.

An upload facility will be specified by the vendor to allow recovery of the download files. Ideally, the source code associated with a download file will be sent to the device in parallel to the actual executable. In those cases, the upload facility provides a mechanism to recover source code in the event other forms are lost.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

7 Layer-F Configuration Management Interface (Informative)

In this version of the O-PAS Standard the Layer-F Configuration Management Interface is non-normative but it is included in the document to show the future direction of the standard.

7.1 Overview

This chapter defines the download/upload services used to download and upload the configuration of an O-PAS conformant component, and to provide control of primary and secondary execution environments. See O-PAS Part 6.2 – Information and Exchange Models: Basic Configuration for the identification of the objects that implement the services.

7.2 Download/Upload File Services

The download/upload file services are provided through a TemporaryFileTransferType, defined in OPC UA Part 5, Clause C.4.

The download/upload file services operate on either the primary environment or the secondary environment within the conformant component:

- The primary environment is defined as the execution environment in which outputs are available and drive physical outputs or provide data to other primary environments
- The secondary environment is defined as an execution environment where outputs are disabled to allow checking of the configuration without impacting the running primary system

GenerateFileForRead is used to start the read file transaction. A successful call of this method creates a temporary FileType object with the file content and returns the NodeId of this object and the file handle to access the object (from OPC UA Part 5, Clause C.4). The first parameter in the GenerateFileForRead method is “generateOptions”. For this use of GenerateFileForRead it is a required parameter with values defined as described in Table 8.

GenerateFileForWrite is used to start the write file transaction. A successful call of this method creates a temporary FileType object and returns the NodeId of this object and the file handle to access the object. (From OPC UA Part 5, Clause C.44.) The first parameter in the method is an optional parameter “generateOptions”. For this use of GenerateFileForWrite, it is defined as described in Table 8.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Table 8: generateOptions Definition

Argument	Description	Data Type
generateOptions	Defines the environment for the configuration file. The values are “0” for the primary environment, the environment where outputs are available, or “1” for the secondary environment where outputs are disabled to allow checking of the configuration without impacting the existing system.	UInt32

7.3 Switch Configuration Environments

SwitchEnvironments is an OPC UA method that is used to switch the configuration environments. A successful call of this method causes the primary environment to become the secondary environment, and the secondary environment to become the primary environment.

Signature

```
SwitchEnvironments(
  );
```

Method Result Codes (defined in Call Service)

Result Codes	Description
Bad_UserAccessDenied	See OPC UA Part 4 for a general description.

Table 9: AddressSpace Representation for the SwitchEnvironments Method

Attribute	Value				
BrowseName	SwitchEnvironmentsAndClose				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
HasProperty	Variable	InputArguments	Argument[]	PropertyType	Mandatory
HasProperty	Variable	OutputArguments	Argument[]	PropertyType	Mandatory

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

8 Layer-F Application Management Interface (Informative)

In this version of the O-PAS Standard the Layer-F Application Management Interface is non-normative but it is included in the document to show the future direction of the standard.

The Layer-F Application Management Interface is defined in this document to control Layer-F applications which have their information and exchange models defined in the other subparts of Part 6.

The Application Management Interface is derived from the example described in OPC UA Part 9, Annex F: System State (Informative) Specification, which is based on OPC UA StateMachines as defined in OPC UA Part 5, Annex B. The Layer-F applications are managed by commands sent to change the state of the execution of the application or application subparts, and by monitoring the state of the execution of the application or application subparts. See the subparts of Part 6 for the identification of the objects that implement the StateMachine.

The following sections define the standard interface of states and methods to be provided by conformant components.

8.1 Application Management States

Each subpart of Part 6 defines the valid states for the application and the location of StateMachines (e.g., at each function block or at each collection of function blocks).

The current state from the StateMachine can be applied to all items in the HasNotifier hierarchy below the object with which the StateMachine is associated, typically at the object representing the application.

There may be multiple ExecutionStateStateMachines deployed, each controlling one or more objects within a conformant component. Figure 5 illustrates the StateMachine representing the ExecutionStates of the application, but does not show all possible transitions.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

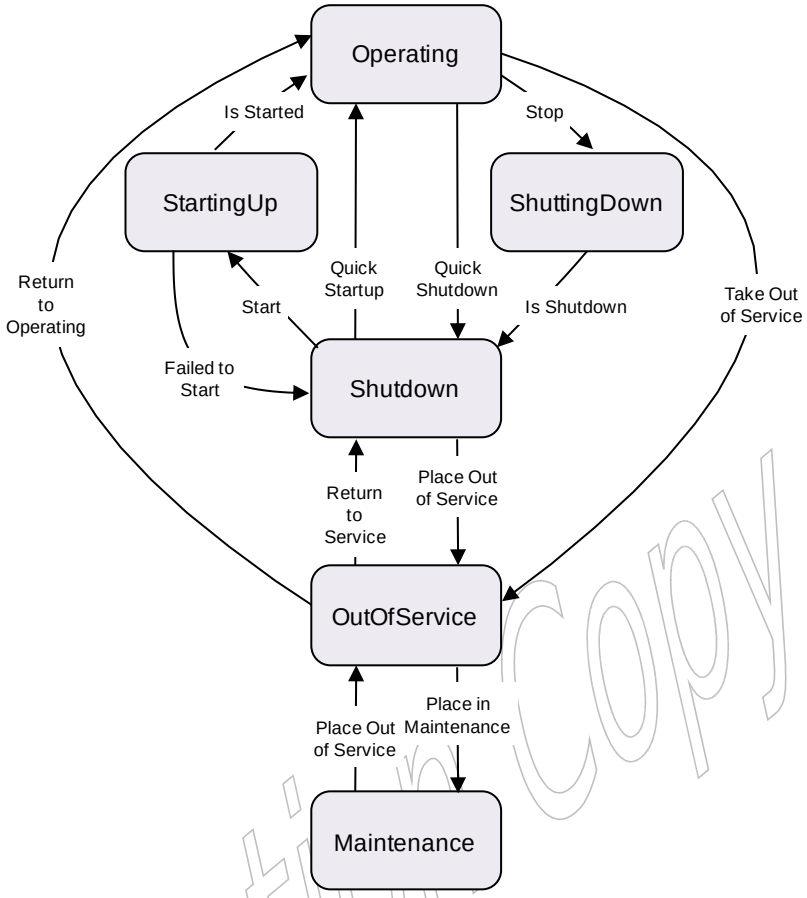


Figure 5: ExecutionState States and Transitions

The states are described as:

- **Operating** – the associated object is performing a programmed function
- **ShuttingDown** – the object is performing a controlled shutdown of its programmed function
- **StartingUp** – the object is performing a controlled startup of its programmed function
- **Shutdown** – the object is not performing its controlled function
- **OutOfService** – the object is not available for use
- **Maintenance** – the object is being controlled in an object-specific manner, typically manually controlled for the purpose of maintenance of the object or the controlled process

The transitions are described as:

- **Start** – an external command to perform a controlled startup of the object's programmed function
- **Is Started** – an internal transition that signifies that the controlled startup of the object's programmed function has completed

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

- **Failed to Start** – an internal transition that signifies that the controlled startup of the object's programmed function was not successfully completed
- **Stop** – an external command to perform a controlled shutdown of the object's programmed function
- **Is Shutdown** – an internal transition that signifies that the controlled shutdown of the object's programmed function has completed
- **PlaceOutOfService** – an external command to place the object out of service
- **ReturnToService** – an external command to return the object to service
- **TakeOutOfService** – an internal transition to take the object out of service, usually due to a detected error condition
- **QuickShutdown** – an external command or internal transition to quickly shut down the object's programmed function; this could leave a controlled process in an unstable or unsafe state
- **QuickStartup** – an external command or internal transition to quickly start up the object's programmed function
- **PlaceInMaintanance** – an external command to place an object in maintenance mode
- **ReturnToOperating** – an external command or internal transition to quickly put the object into the operating state from the out of service state

8.2 ExecutionStateStateMachineType

The ExecutionStateStateMachineType includes a hierarchy of sub-states. It supports multiple transitions between Operating, StartingUp, ShuttingDown, Shutdown, OutOfService, and Maintenance.

The ExecutionStateStateMachineType is illustrated in Figure 6.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

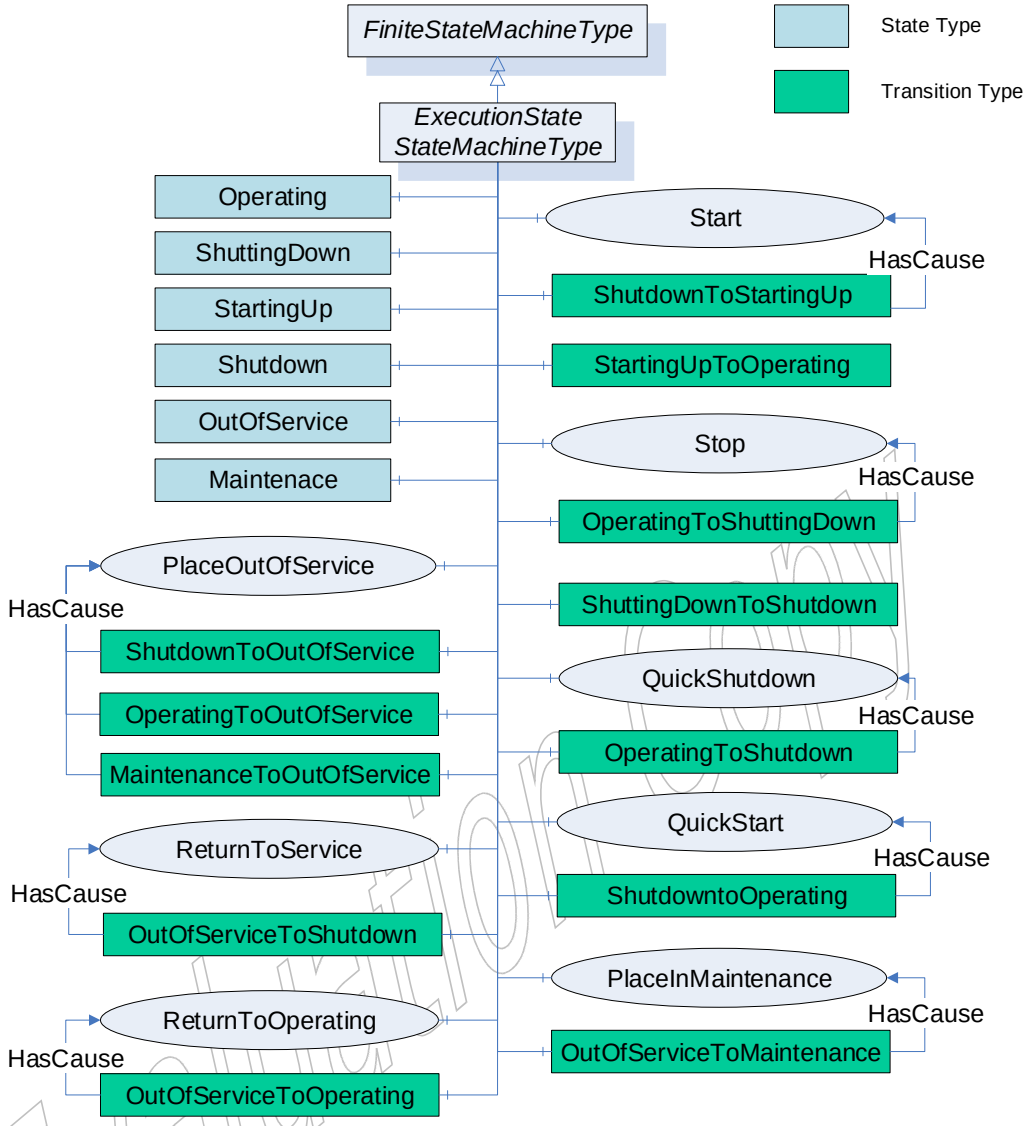


Figure 6: ExecutionStateStateMachineType Model

The ExecutionStateStateMachineType is formally defined in Table 10.

Table 10: ExecutionStateStateMachineType Definition

Attribute	**Value**				
BrowseName	ExecutionStateStateMachineType				
IsAbstract	False				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
Sub-type of the FiniteStateMachineType defined in OPC UA Part 5.					
HasComponent	Object	Operating		0:StateType	

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

HasComponent	Object	ShuttingDown		0:StateType	
HasComponent	Object	StartingUp		0:StateType	
HasComponent	Object	Shutdown		0:StateType	
HasComponent	Object	OutOfService		0:StateType	
HasComponent	Object	Maintenance		0:StateType	
HasComponent	Object	ShutdownToStartingUp		0:TransitionType	
HasComponent	Object	StartingUpToOperating		0:TransitionType	
HasComponent	Object	OperatingToShuttingDown		0:TransitionType	
HasComponent	Object	ShuttingDownToShutdown		0:TransitionType	
HasComponent	Object	OperatingToShutdown		0:TransitionType	
HasComponent	Object	ShutdownToOperating		0:TransitionType	
HasComponent	Object	OutOfServiceToMaintenance		0:TransitionType	
HasComponent	Object	ShutdownToOutOfService		0:TransitionType	
HasComponent	Object	OperatingToOutOfService		0:TransitionType	
HasComponent	Object	OutOfServiceToShutdown		0:TransitionType	
HasComponent	Object	OutOfServiceToOperating		0:TransitionType	
HasComponent	Object	MaintenanceToOutOfService		0:TransitionType	
HasComponent	Method	Start	Defined in Section 8.3.2		Optional
HasComponent	Method	Stop	Defined in Section 8.3.3		Optional
HasComponent	Method	QuickShutdown	Defined in Section 8.3.4		Optional

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

HasComponent	Method	QuickStartup	Defined in Section 8.3.5		Optional
HasComponent	Method	PlaceInMaintenance	Defined in Section 8.3.6		Optional
HasComponent	Method	PlaceOutOfService	Defined in Section 8.3.7		Optional
HasComponent	Method	ReturnToService	Defined in Section 8.3.8		Optional
HasComponent	Method	ReturnToOperating	Defined in Section 8.3.9		Optional

Note: The actual selection of states and transitions would depend on the deployment of the StateMachine. If the StateMachine were being applied to an object controlling a process unit or other part of a process, it might have a different set of states than if it was applied to a single function block. The single function block may only have Operating, OutOfService, and Maintenance, while the process unit may have all of the described states and transitions.

The StateMachine supports six possible states including: Operating, ShuttingDown, StartingUp, Shutdown, OutOfService, and Maintenance. It supports twelve possible transitions and nine possible methods.

Table 11: ExecutionStateStateMachineType Transitions

BrowseName	References	BrowseName	TypeDefinition
Transitions			
ShutdownToOperating	FromState	Shutdown	0:StateType
	ToState	Operating	0:StateType
	HasCause	QuickStart	Method
OperatingToShutdown	FromState	Operating	0:StateType
	ToState	Shutdown	0:StateType
	HasCause	QuickShutdown	Method
ShuttingdownToShutdown	FromState	ShuttingDown	0:StateType
	ToState	Shutdown	0:StateType
OperatingToShuttingdown	FromState	Operating	0:StateType
	ToState	ShuttingDown	0:StateType
	HasCause	Stop	Method

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

BrowseName	References	BrowseName	TypeDefinition
StartingUpToOperating	FromState	StartingUp	0:StateType
	ToState	Operating	0:StateType
ShutdownToStartingUp	FromState	Shutdown	0:StateType
	ToState	StartingUp	0:StateType
	HasCause	Start	Method
OutOfServiceToShutdown	FromState	OutOfService	0:StateType
	ToState	Shutdown	0:StateType
	HasCause	OutOfServiceShutdown	Method
ShutdownToOutOfService	FromState	Shutdown	0:StateType
	ToState	OutOfService	0:StateType
	HasCause	OutOfServiceShutdown	Method
OutOfServiceToOperating	FromState	OutOfService	0:StateType
	ToState	Operating	0:StateType
	HasCause	PlaceOutOfService	Method
OperatingToOutofService	FromState	Operating	0:StateType
	ToState	OutOfService	0:StateType
	HasCause	PlaceOutOfService	Method
MaintenanceToOutofService	FromState	Maintenance	0:StateType
	ToState	OutOfService	0:StateType
	HasCause	Maintain	Method
OutOfServiceToMaintenance	FromState	OutOfService	0:StateType
	ToState	Maintenance	0:StateType
	HasCause	Maintain	Method

8.3 ExecutionStateStateMachine Methods

8.3.1 ExecutionState Command Method Template

This section defines the template for the methods used to command an ExecutionStateStateMachine. The method template defines no input or output parameters and has no referenced objects or variables.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Signature

```
<<Method Name>>(
  );
```

Method result codes are defined in Table 12.

Table 12: Method Result Codes

Result Codes	Description
Bad_MethodInvalid	See OPC UA Part 4 – Services for the description of this result code. (The method ID does not refer to a method for the specified object.)
Bad_NotImplemented	See OPC UA Part 4 – Services for the description of this result code. (Requested operation is not implemented.)
Bad_NodeIdUnknown	See OPC UA Part 4 – Services for the description of this result code. (Used to indicate that the specified object is not valid.)
Bad_InvalidState	See OPC UA Part 4 – Services for the description of this result code. (The operation cannot be completed because the object is closed, uninitialized, or in some other invalid state.)

8.3.2 Start Method

The Start method is an implementation of the ExecutionState command method template defined in Section 8.3.1. It allows an OPC UA client to command the StateMachine to enter the Starting state.

Signature

```
Start (
    TBD
  );
```

8.3.3 Stop Method

The Stop method is an instance implementation of the ExecutionState command method template defined in Section 8.3.1. It allows an OPC UA client to command the StateMachine to enter the Stopping state.

Signature

```
Stop (
    TBD
  );
```

8.3.4 QuickShutdown Method

The QuickShutdown method is an instance implementation of the ExecutionState command method template defined in Section 8.3.1. It allows an OPC UA client to command the StateMachine to enter the Shutdown state.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Signature

```
QuickShutdown (
    TBD
  );
```

8.3.5 QuickStartup Method

The QuickStartup method is an instance implementation of the ExecutionState command method template defined in Section 8.3.1. It allows an OPC UA client to command the StateMachine to enter the Operating state.

Signature

```
QuickStartup (
    TBD
  );
```

8.3.6 PlaceInMaintenance Method

The PlaceInMaintenance method is an instance implementation of the ExecutionState command method template defined in Section 8.3.1. It allows an OPC UA client to command the StateMachine to enter the Maintenance state.

Signature

```
PlaceInMaintenance (
    TBD
  );
```

8.3.7 PlaceOutOfService Method

The PlaceOutOfService method is an instance implementation of the ExecutionState command method template defined in Section 8.3.1. It allows an OPC UA client to command the StateMachine to enter the OutOfService state.

Signature

```
PlaceOutOfService (
    TBD
  );
```

8.3.8 ReturnToService Method

The ReturnToService method is an instance implementation of the ExecutionState command method template defined in Section 8.3.1. It allows an OPC UA client to command the StateMachine to enter the Shutdown state.

Signature

```
ReturnToService (
    TBD
  );
```


© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

8.3.9 ReturnToOperating Method

The ReturnToOperating method is an instance implementation of the ExecutionState command method template defined in Section 8.3.1. It allows an OPC UA client to command the StateMachine to enter the Operating state.

Signature

```
ReturnToOperating (
    TBD
  );
```

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

9 O-PAS Profiles and Facets

9.1 Overview

Profiles and facets are named groupings of conformance requirements to define the functionality required of a conformant component.

9.2 Profiles and Facets

This section describes the profiles and facets that are provided as part of the O-PAS information model for portable configurations.

9.2.1 OBC-F-101: O-PAS Package File Facet

An O-PAS conformant engineering tool that claims support for OBC-F-101 shall support the O-PAS import/export requirements defined in Table 13.

Table 13: OBC-F-101: O-PAS Package File Facet Definition

Facet	Description	Reference
OBC-F-101	O-PAS Package File	Products that implement this facet shall support the conformance requirements in the following sections of this document: • Cybersecurity (Chapter 4) • Import/Export (.opas) File (Section 5.2) • Conformance Units (Section 5.2) • Part-Properties XML (Appendix A) • OPC UA in AML (Appendix E)

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

A O-PAS Part-Properties (Normative)

O-PAS Part-Properties allows users to specify metadata for parts (such as source files) within a .opas package. There is only one O-PAS Part-Properties part in each package; all properties for all parts are stored in it.

Table 14 shows the metadata for a part that can be stored in the O-PAS Part-Properties. All elements listed in the table can be included or omitted as tools see fit, but no additional elements shall be added.

An O-PAS conformant engineering tool intended to store additional information in the package shall use CustomInfoType elements with a unique ID, preferably a URI. [CRQ-P6.1-0071]

Table 14: O-PAS Part-Properties for a Part

Metadata	Description
description	A description of what is in the part.
version	The version of the part; e.g., a build number.
created	When this part was initially created.
lastModified	When this part was last modified.
author	Who last edited this part.
customInfo	Elements used to store additional, tool-specific information about the part.

The propertiesVersion element in CT_PartProperties shall always be set to “1.0”. [CRQ-P6.1-0053]

A.1 XML Schema for O-PAS Part-Properties

The O-PAS Part-Properties part shall follow the XSD schema defined below. [CRQ-P6.1-0054]

```
<?xml version = "1.0" encoding = "UTF-8"?>
<xsd:schema     xmlns:xsd            = "http://www.w3.org/2001/XMLSchema"
                targetNamespace      = "http://www.opengroup.org/open-process-
atiomation/O-PAS/part-properties"
                xmlns                = "http://www. opengroup.org/open-process-
atiomation/O-PAS/part-properties "
                elementFormDefault   = "qualified"
                attributeFormDefault = "unqualified">

<!-- Global Elements -->
  <xsd:element name = "partProperties"      type = "CT_PartProperties" />

<!-- Base Object Types  -->
  <xsd:complexType name = "CT_PartProperties">
```


© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

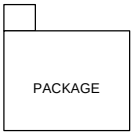
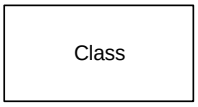
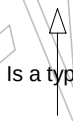
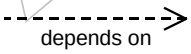
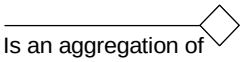
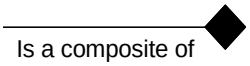
```
  <xsd:sequence>
    <xsd:element name = "propertiesVersion" type = "xsd:string" />
    <xsd:element name = "fileInfo"        type = "CT_FileInfoType"
                                          minOccurs = "0"
                                          maxOccurs = "unbounded"/>
  </xsd:sequence>
</xsd:complexType>

<xsd:complexType name = "CT_FileInfoType">
  <xsd:sequence>

    <xsd:attribute ref="relId"/>
    <xsd:element name = "description"     type = "xsd:string"
                                          minOccurs = "0"
                                          maxOccurs = "unbounded"/>
    <xsd:element name = "version"         type = "xsd:string"
                                          minOccurs = "0"/>
    <xsd:element name = "created"         type = "xsd:dateTime"
                                          minOccurs = "0" />
     <xsd:element name = "lastModified"   type = "xsd:dateTime"
                                          minOccurs = "0" />

    <xsd:element name = "author"          type = "xsd:string"
                                          minOccurs = "0"
                                          maxOccurs = "unbounded"/>
    <xsd:element name = "customInfo"      type = "CT_customInfoType"
                                          minOccurs = "0"
                                          maxOccurs = "unbounded"/>
  </xsd:sequence>
</xsd:complexType>

<xsd:complexType name = "CT_CustomInfoType">
  <xsd:sequence>
    <xsd:element name = "ID"              type = "xsd:string" />
    <xsd:element name = "description"     type = "xsd:string"
                                          minOccurs = "0"
                                          maxOccurs = "unbounded"/>
    <xsd:element name = "customData"      type = "xsd:string"
                                          minOccurs = "0"
                                          maxOccurs = "unbounded"/>
  </xsd:sequence>
</xsd:complexType></xsd:schema>

<xsd:simpleType name="ST_RelationshipId">
  <xsd:restriction base="xsd:ID"/>
</xsd:simpleType>
<xsd:attribute name="relId" type="ST_RelationshipId"/>
```


© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

B UML (ISO/IEC 19501) Model Representations (Informative)

Implementation-independent object models are often depicted using the Unified Modeling Language™ (UML) notational methodology, as defined in ISO/IEC 19501. For O-PAS implementers that may not be familiar with UML Class Models, Table 15 provides a brief overview of the UML notations used in UML Class Models.

Table 15: UML Notation for Class Models

Symbol	Definition
	Defines a package, a collection of object models, state models, use-cases, and other UML models. Packages are general-purpose grouping mechanisms used to organize semantically-related model elements. In this document, a package is used to specify an external model, such as a production rule model, or a reference to another part of the model.
	Represents a UML class of objects, each with the same types of attributes. Each object is uniquely identifiable or enumerable. In the O-PAS models no operations or methods are listed for the classes and attributes typically defined in associated tables.
	An association between elements of a class and elements of another or the same class. Each association is identified. May have the expected number or range of members of the subclass, when "*" indicates an indeterminate number. For example, 0.* means that zero or more members of the subclass may exist. AssociationName, which is a verb, can have an arrow beside it to indicate in which direction it operates. For example, a verb operating from left (subject) to right (object) would have an arrow like this ► to its right.
	Generalization (arrow points to the super class) shows that an element of the class is a specialized type of the super class.
	Dependence is a weak association that shows that a modeling element depends on another modeling element. The item at the tail depends on the item at the head of the relationship.
	Aggregation (made up of) shows that an element of the class (at the diamond end) is made up of elements of other classes.
	Composite shows a strong form of aggregation, which requires that a part instance be included in at most one composite at a time and that the composite object (at the filled diamond end of the line) has sole responsibility for disposition of its parts.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

C Mapping UML to the AML (Informative)

The following general rules (illustrated in Figure 7) can be used to map a UML Class Model into an AML representation:

- If a UML object is an abstract class, then a role class is defined for the object
- If the UML object is intended to represent an O-PAS object (such as a standard function block, or a standard signal type), then an SUC is defined for the object
- If the UML object is intended to represent an object which can be used as the parent type of a vendor-specific sub-type, then a role class should be defined for the object
- If a UML object is contained (aggregation) within another object, then no interface class is needed for the relationship; however, a conformance rule must be stated that the child object must exist someplace under the parent object (such as in another container object [folder]) – this allows the application to always find the child object
- If a UML object is a sub-type (derived type) of another object, then use the "Class Reference" in the header to store the reference to the base class
- If there is an association between objects, then this is represented as an interface class representing the specific association
- If one instance of a UML object is aggregated by multiple other UML objects, it would be instantiated as an internal element in the InstanceHierarchy for one of the UML objects and have mirror objects in the InstanceHierarchies for the remaining other UML objects

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

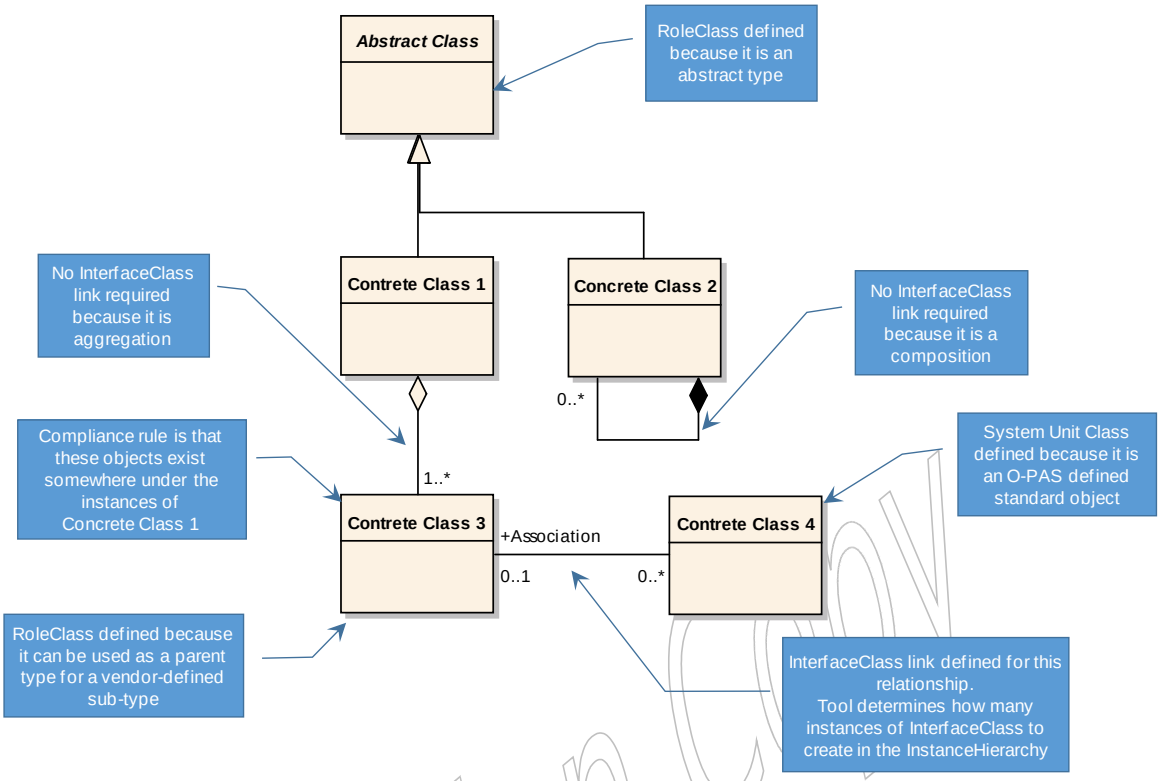


Figure 7: General Rules for Mapping the UML Class Model to the AML

Each subpart of Part 6 defines the following AML components that are used to define the structure of the exchanged InstanceHierarchy in an AML file:

- An Attribute Type Library (AttributeTypeLib)

  This describes common attributes that are used in the other SUC and role class libraries.

- An Interface Class Library (InterfaceClassLib)

  This describes the classes of internal interfaces (links) between elements in the InstanceHierarchy. These are used to represent links between instance objects in the object model. This also defines the classes of external interfaces (references) to information not covered in the CAEX-based model, such as a source file for a function block program.

- A Role Class Library (RoleClassLib)

  This describes the class of abstract functionality, which may be implemented in many different SUCs.

- An SUC Library (System Unit ClassLib)

  This describes the SUCs that are reusable templates for creating instances in the InstanceHierarchy. The SUCs are used to construct the InstanceHierarchy by the configuration tools.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

D OPC UA (IEC 62541) Model Representations (Informative)

An introduction to OPC UA can be found in O-PAS Part 4 – O-PAS Connectivity Framework (OCF).

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

E Using AML Configuration Files to Specify OPC UA Nodes (Normative)

Specifying a specific OPC UA node is a common requirement in configuring OPC UA objects. This is difficult to do when the node does not yet exist; e.g., as occurs when trying to configure alarm objects using the AML.

To overcome these forward references, OPC UA-specific AML attribute types, role classes, and interface classes have been created and placed in an attribute type library called: O-PAS-UA-NodeAttributeTypeLib, an interface class library called: O-PAS-UA-NodeInterface ClassLib, and a role class library called: O-PAS-UA-NodeRoleClassLib.

A conformant engineering tool shall import and use the O-PAS-UA-NodeAttributeTypeLib. [CRQ-P6.1-0055]

A conformant engineering tool shall import and use the O-PAS-UA-NodeInterfaceClassLib. [CRQ-P6.1-0056]

A conformant engineering tool shall import and use the O-PAS-UA-NodeRoleClassLib. [CRQ-P6.1-0057]

A conformant engineering tool shall use the required standard O-PAS-UA libraries to interface to any embedded OPC UA server. [CRQ-P6.1-0058]

These types include:

- UA-NodeIdType (attribute type) which describes a namespace and NodeId format

  It supports four approaches to specifying the desired OPC UA NodeId, but only one of them shall be used in any particular NodeId reference. The others should be omitted from the nodeset sent to a DCN during configuration download.

- UA-FindAliasType (attribute type) which can be used to provide information to an OPC UA server to find the OPC UA NodeId of the specified alias

- UA-NodeAbsolute (role class) which combines the previous two attributes to provide the information needed to support all the functionality OPC UA offers to find OPC UA NodeIds

- UA-NodeRelative (role class) which can be used to build AML InstanceHierarchies analogous to an OPC UA node hierarchy

- UA-VariableConnection (interface class) which is used in the above role classes to act as the source of an internal link going to the AML element that requires being linked to the UA node for configuration

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

E.1 AML Definitions

This section defines the AML role types, attribute types, and interface classes needed to express UA nodes in AML files. The AML can either be linked directly to a particular UA node (absolute) or they can be linked by using their location in the InstanceHierarchy relative to an element that is linked to a UA node (relative). In the latter case, the InstanceHierarchy in the AML replicates the structure of the UA Information Model and UA BrowsePaths can be derived from the AML InstanceHierarchy.

E.1.1 UA-NodeIdType Attribute Type

The UA-NodeIdType AML attribute type contains these attributes:

Name	Attribute Type	M/O/E	Comment
NamespaceURI	xs:anyURI	O	Namespace qualifier for the NodeId. RefSemantic set to: OPC UA Optional. If omitted, the Local Server URI (i.e., namespace index 1; see OPC UA Part 200) is assumed.
Include exactly one of the following:			RefSemantic for all four set to: OPC UA – only 1 of the 4 NodeId types
NumericNodeId	Int32	O	Numeric representation of a NodeId.
StringNodeId	string	O	String representation of a NodeId.
GUIDNodeId	string	O	GUID representation of a NodeId in standard GUID format. See OPC UA Part 6.
OpaqueNodeId	xs:base64Binary	O	NodeId in a format that cannot be decoded by a client.

E.1.2 UA-AliasName Attribute Type

The UA-AliasName AML attribute type is used to store OPC UA AliasNames in the AML (see OPC UA Part 17). It is of DataType xs:string and has no sub-attributes.

E.1.3 UA-FindAliasType Attribute Type

The UA-FindAliasType AML attribute type provides a DCN's OPC UA server with the information required to call the *FindAlias*() method to get a list of NodeIds that match.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Table 16: UA-FindAliasType Attribute Type Definition

Name	Attribute Type	M/O/E	Comment
AliasName	UA-AliasName	M	A string that can contain wildcards, used to find a list of AliasNames (see OPC UA Part 4 “Wildcard Characters” table for supported wildcards).
ReferenceTypeFilter	NodeId	O	A NodeId that represents a ReferenceType (i.e., alias for one of its sub-types) that restricts the search. Any ReferenceType includes all sub-types of that ReferenceType. RefSemantic set to: OPC UA Optional

E.1.4 UA-BrowsePath Attribute Type

The UA-BrowsePath AML attribute type is used to express an OPC UA BrowsePath.

Table 17: UA-BrowsePath Attribute Type Definition

Name	Attribute Type	M/O/E	Comment
BrowsePathNamespaceURIs	String[]	M	The array must have the same length as BrowsePath. An element with an empty string should be substituted with the NamespaceURI of the root node from which the BrowsePath is followed. An element with the string “1” should be substituted with the Local Server URI (namespace index 1).
BrowsePath	String[]	M	Semantics: Non-empty: Array must not be empty
ReferenceTypeId	NodeId[]	O	If present the array must have the same length as BrowsePath. Each element contains the NodeId of the UA ReferenceType to follow. If missing, each element is assumed to be the Aggregates reference (OPC UA Part 13). RefSemantics set to: OPC UA Optional

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Name	Attribute Type	M/O/E	Comment
isInverse	Boolean[]	O	If present the array must have the same length as BrowsePath. Only inverse references shall be followed if the element value is TRUE. Only forward references shall be followed if the element value is FALSE. If missing, each element is assumed to be FALSE. RefSemantics set to: OPC UA Optional
includeSubtypes	Boolean[]	O	If present the array must have the same length as BrowsePath. Each element indicates whether sub-types of the ReferenceType should be followed. Sub-types are included if this value is TRUE. If missing, each element is assumed to be TRUE. RefSemantics set to: OPC UA Optional

E.1.5 UA-Description Attribute Type

The UA-Description attribute type is based on the standard AML attribute type LocalizedAttribute and can be used to configure the UA attribute “Description”. The data type is xs:string and the semantics are set to: “UA Optional: If present, the value of the attribute is used to fill the “Description” attribute of the UA variable”.

E.1.6 UA-NodeAbsolute Role Class

The UA-NodeAbsolute role class provides the information needed to support all the functionality OPC UA offers to find OPC UA NodeIds. This information can be used to match the parameters in the TranslateBrowsePathsToNodeIds Service defined in OPC UA Part 4. The editor building the AML should complete the mix of attributes required by the translation service to be used. See the examples in Section E.1.11.

Either NodeId or alias must be provided (but never both) and are marked “E” for exclusive. All other fields are optional. M signifies that the attribute is Mandatory; O signifies that the attribute is Optional.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Table 18: UANodeTargetInterface Interface Class Attributes Definition

Name	Attribute Type	M/O/E	Comment
DiscoveryEndpoint	string	O	The discovery endpoint URL of the UA server containing the desired node or of the discovery server to begin the search. DiscoveryEndpoint allows access to discovery services without a session and without message security. RefSemantic set to: OPC UA Optional. If the DiscoveryEndpoint is omitted, then the search for the ServerURI or alias will begin using the DiscoveryEndpoint of some preconfigured Global Discovery Server (GDS).
ServerURI	xs:anyURI	O	Uniquely identifies an OPA UA server instance in a “system”. RefSemantic set to: OPC UA Optional. If the ServerURI is omitted, it is expected that the DiscoveryEndpoint is that of a single server that contains the desired node.
NodeId	NodeId	E	The root node from which the BrowsePath is followed. If NodeId is present but the DiscoveryEndpoint and ServerURI are omitted, the NodeId is presumed to be local to the server being configured via the AML.
Alias	FindAlias	E	An alternate way to specify the root node. If ServerURI is present, only that specific server will be used to find the alias.
BrowsePath	UA-BrowsePath	O	Semantics: OPC UA Optional

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

E.1.7 UA-VariableAbsolute Role Class

This role class derives from the UA-NodeAbsolut role class. It shall be used instead of UA-NodeAbsolute if the targeted UA node is a UA variable. It adds a UA-VariableConnection interface with direction Out that allows linking consumers of the variable.

E.1.8 UA-NodeRelative Role Class

This role is intended to be applied to internal elements of AML elements (here called “known elements”) in the InstanceHierarchy which are representing OPC UA nodes (here called “known node”) that have a known runtime location. The runtime location of the known element can, for example, be described using the UA-NodeAbsolute role. There are more mechanisms than just UA-NodeAbsolute to have AML elements connected to known/findable UA nodes.

Internal elements of such a known element can now have the UA-NodeRelative role as a role requirement to indicate that this internal element (here called “sub-element”) represents a UA node (here called “sub-node”) that is the target of a hierarchical reference from the known bode. This is done using the name of the internal element as the BrowseName for a UA BrowsePath from the known node to the sub-node. The NamespaceURI of the sub-element completes the information needed for the one-segment BrowsePath.

Table 19 shows the attributes of this role class. In addition this role class contains an external interface of UA-VariableConnection type called “ConnectionProvider” with the “Direction” attribute set to “Out”.

Table 19: Attributes for UA-NodeRelative Role Class

Name	Attribute Type	M/O/E	Comment
BrowsePathSegmentURI	xs:anyURI	O	This attribute specifies the NamespaceURI for the BrowsePath segment represented by this element. The one-segment BrowsePath is built from the element name and the NamespaceURI in this attribute. Semantics: If omitted, the local server namespace (1) is used.
Description	UA-Description	O	This attribute can be used to configure the UA node attribute “Description”. Semantics: UA Optional: If present, the value of the attribute is used to fill the “Description” attribute of the UA variable.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

E.1.9 UA-VariableRelative Role Class

This role class derives from UA-NodeRelative and shall be used if the (internal) element with this role represents a UA variable. Table 20 defines and describes the attributes of this role class. In addition this role class contains an external interface of UA-VariableConnection type called “ConnectionProvider” with the “Direction” attribute set to “Out”.

Table 20: Attributes for UA-VariableRelative Role Class

Name	Attribute Type	M/O/E	Comment
BrowsePathSegmentURI	xs:anyURI	O	This attribute specifies the NamespaceURI for the BrowsePath segment represented by this element. The one-segment BrowsePath is built from the element name and the NamespaceURI in this attribute. Semantics: If omitted, the local server namespace (1) is used.
Description	UA-Description	O	This attribute can be used to configure the UA node attribute “Description”. Semantics: UA Optional: If present, the value of the attribute is used to fill the “Description” attribute of the UA variable.

E.1.10 UA-VariableRelativeConfigurable Role Class

This role class derives from UA-VariableRelative and shall be used if the AML is used to configure a value for this UA variable node. Table 21 defines and describes the attributes of this role class.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Table 21: Attributes for UA-VariableRelativeConfigurable Role Class

Name	Attribute Type	M/O/E	Comment
BrowsePathSegmentURI	xs:anyURI	O	This attribute specifies the NamespaceURI for the BrowsePath segment represented by this element. The one-segment BrowsePath is built from the element name and the NamespaceURI in this attribute. Semantics: If omitted, the local server namespace (1) is used.
Description	UA-Description	O	This attribute can be used to configure the UA node attribute "Description". Semantics: UA Optional: If present, the value of the attribute is used to fill the "Description" attribute of the UA variable.
Value		M	The value stored for the targeted UA variable node.

E.1.11 UA-NodeConnection Interface Class

The UA-NodeConnection interface class derives from the standard AML interface class "Signal". It is used to connect users (e.g., alarms, HMIs) of OPC UA nodes to the host of UA nodes via internal links in the AML. The tool for the user side follows the internal link to the host side. The AML element containing the interface on the host side provides all the information needed for the consumer to find that UA node at runtime.

The attributes of the interface class are formally defined in Table 22.

On the user side the "Direction" attribute shall have the value "In", and on the host side the "Direction" attribute shall have the value "Out". Intermediate connection points, such as a TagList, shall use a value of "InOut". Internal links shall always connect from an interface with "Out" direction to one with "In" direction, or from "Out" to "InOut" or "InOut" to "In", and there shall be only one internal link to/from an interface with "In" direction, and an "InOut" can only be connected to one "Out". All internal links to/from this interface must go to another interface of this class or a subclass thereof.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Table 22: Attributes for UA-NodeConnection Interface Class

Name	Attribute Type	M/O/E	Comment
Direction	Direction	M	The direction of the interface. “Out” means it provides/hosts a UA node. “In” means consumes a UA node (e.g., as an alarm source). “InOut” is used on intermediate steps in the link, such as the TagList.

E.1.12 UA-VariableConnection Interface Class

The UA-VariableConnection interface class derives from UA-NodeConnection. It is used to connect consumers (subscribers/clients) of OPC UA variables to producers (publishers/servers) of UA variables via internal links in the AML. The tool for the consumer side follows the internal link to the producer side. The AML element containing the interface on the producer side provides all the information needed for the consumer to create a UA subscription to the variable at runtime.

The attributes of the interface class are formally defined in Table 23.

On the consumer side the “Direction” attribute shall have the value “In”, and on the producer side the “Direction” attribute shall have the value “Out”. Intermediate connection points, such as a TagList, shall use a value of “InOut”. Internal links shall always connect from an interface with “Out” direction to one with “In”, or from “Out” to “InOut” or “InOut” to “In”, and there shall be only one internal link to/from an interface with “In” direction, and an “InOut” can only be connected to one “Out”. All internal links to/from this interface must go to another interface of this class or a subclass thereof.

Table 23: Attributes for UA-VariableConnection Interface Class

Name	Attribute Type	M/O/E	Comment
Direction	Direction	M	The direction of the interface. “Out” means it provides/produces a UA variable. “In” means it consumes a UA variable (usually a subscription). “InOut” is used on intermediate steps in the link, such as the TagList.

E.2 Example Usage of UA-NodeAbsolute (Informative)

E.2.1 Target is a Variable with an Alias

This simplest use of the UA-NodeAbsolute is to provide the single field AliasName and omit all other fields.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

For example, given that "TT101" is a global OPC UA alias for the OPC UA variable representing the desired temperature in the process, setting AliasName is all that is necessary to allow an OPC UA server receiving the AML to find the NodeId of the temperature.

```
AliasName = "TT101"
```

E.2.2 Target has a Defined Path from a Node with an Alias

Given that:

- A global OPC UA alias "PID202" exists for the OPC UA object representing a particular PID block
- The desired target is the setpoint of the PID block
- The PID block is the standard O-PAS PID block
- The BrowseName for the O-PAS PID block's setpoint parameter is defined in the O-PAS Standard as "SP"

The NodeId can be found by an OPC UA server receiving an AML file that contains this information:

Attribute	Value	Commentary
AliasName	"PID202"	Alias for a PID block.
BrowsePath[0]	"SP"	The O-PAS Standard name for a setpoint in a PID block.
BrowsePathNamespaceURIs[0]	https://www.opengroup.org/open-process-automation/O-PAS/CF	The O-PAS Standard namespace as given in O-PAS Part 4.

E.2.3 Target is a Well-Known Node Defined in a Specification

When configuring an OPC UA alarm, a ConditionClassId is mandatory for each alarm instance per OPC UA Part 9.

The OPC UA specifies the standard ConditionClassId as ProcessConditionClassType and that node is given a well-known NodeId (0:11164 per the standard OPC UA nodeset) found in every OPC UA alarms and conditions server.

With this knowledge, the UA-NodeAbsolute for ConditionClassId can be set to:

- NamespaceURI = "http://opcfoundation.org/UA/"
- NumericNodeId = 11164

E.2.4 Target is an Instance Node in a Server with a Known DNS Name

Given that:

- The target is an instance node in an OPC UA server known by a DNS name
- The target has a known GUID NodeId in that server

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

The AML configuration file can reference the node by filling in the DiscoveryEndpoint and NodeId attributes's GUIDNodeId as follows:

- DiscoveryEndpoint = "opc.tcp://server42.baytown.xom.com:4840"
- GUIDNodeId = "e2643ad2-ba7c-40b4-b319-1aa2901447ae"

Note that, in this example, the NodeId's NamespaceId is not used; i.e., is not sent to the OPC UA server. Therefore, the OPC UA server knows to use its own instance namespace; i.e., Namespace Index 1.

E.3 Example of UA-NodeRelative (Informative)

This example is built on the OPC UA Information Model of a house shown in Figure 8. It contains a House object, which contains a LivingRoom, which has a MainLight that can be On or Off. The MainLight is configurable with an automatic timer that turns it off *x* minutes after it was turned on (AutoOffTimer). For this example, the number of minutes is configurable in the AML file and the House UA object is assumed to be known (how exactly is not relevant for the example). The namespace used for all BrowsePaths is "my.hou.se/ua".

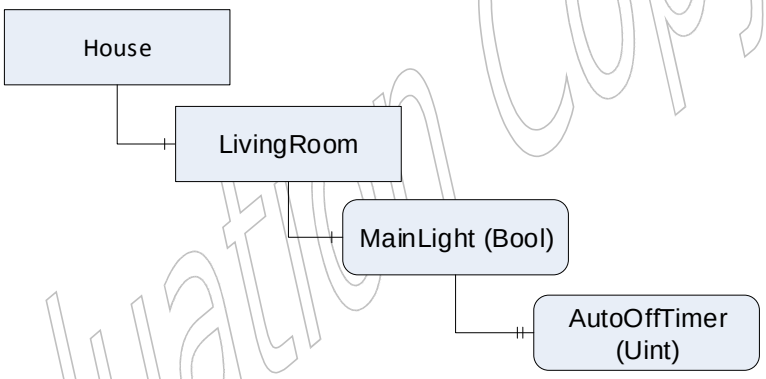


Figure 8: Example UA Information Model for Showing Sub-Nodes in the AML

An overview of an AML InstanceHierarchy that can be created for this example is shown in Figure 9 and attribute values are provided in Table 24. The House element is connected to a known UA node. How exactly is irrelevant for this example; we simply assume it is known.

The House element has an internal element with the UA-NodeRelative role that represents the LivingRoom which in turn has an internal element with the UA-VariableRelative role called MainLight. The name of the internal elements and their BrowsePathSegmentURI attributes are used to form a BrowsePath from House to LivingRoom and from LivingRoom to MainLight. Note how LivingRoom does not use UA-VariableRelative, but rather the more generic UA-NodeRelative. This is because it is a UA object, not a UA variable.

Similarly, MainLight is using UA-VariableRelative and not UA-VariableRelativeConfigurable because it is used for hierarchical purposes and setting a configured value for this node in the AML is not desired in this example.

The last step is again an internal element, this time inside the MainLight element. This last element is for the AutoOffTimer and its name and attribute again build a BrowsePath from the MainLight UA node. The AutoOffTimer element has the role UA-VariableRelativeConfigurable

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

and allows configuration of a value for the AutoOffTimer OPC UA variable of MainLight in LivingRoom of House.

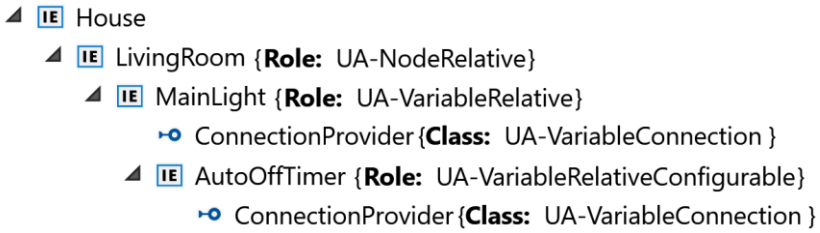


Figure 9: Example of Possible AML InstanceHierarchy for Showing Sub-Nodes in the AML

Table 24: Attributes for Example of Possible AML InstanceHierarchy for Showing Sub-Nodes in the AML

Element	Attribute	Value	Comment
House/LivingRoom	BrowsePathNamespaceURI	<Omitted>	Attribute is omitted to indicate the Local Server Namespace #1.
House/LivingRoom/MainLight	BrowsePathSegmentURI	My.hou.se/ua	URI complementing “MainLight” (element name) to build the BrowsePath from the LivingRoom node to the MainLight UA variable.
House/LivingRoom/MainLight/ConnectionProvider	Direction	Out	Indicate that this element represents a UA variable that is provided for others to subscribe to.
House/LivingRoom/MainLight/AutoOffTimer	BrowsePathSegmentURI	My.hou.se/ua	URI complementing “AutoOffTimer” (element name) to build the BrowsePath from the MainLight node to the AutoOffTimer UA variable.
House/LivingRoom/MainLight/AutoOffTimer	Value	15	Value that “AutoOffTimer” should be set to. In this case it is set to 15 minutes.
House/LivingRoom/MainLight/AutoOffTimer/ConnectionProvider	Direction	Out	Indicate that this element represents a UA variable that is provided for others to subscribe to.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

F Common Relationships between AML, OPC UA Nodeset, and Code Files in a .opas Package (Informative)

It is very common for O-PAS projects to contain applications that make use of custom user or third party-defined code to provide functionality. Such code often contains reusable units such as function blocks or classes. Since the O-PAS Standard uses OPC UA as the protocol for applications to provide information to external components such as HMIs, alarm engines, or historians, it is desirable to have a UA object type definition that reflects the function block or class.

These UA object types might include additional information to what is in the code. For example, they can group variables in the code into functional groups with non-hierarchical references or add instance-specific information such as value precision and descriptions that, while useful to an HMI, are irrelevant to the code itself.

In this appendix, a mechanism is described for how to store code files (source code) and UA type definitions (UA nodesets) accompanying code in a .opas package. While it is only informative in this document, other O-PAS subparts can utilize this mechanism to create normative definitions for how they integrate their source files into .opas packages.

F.1 Overview of the Package Contents and Use

For the use-case described above, three things are needed in the .opas package:

1. The AML file
2. One or more code files
3. One or more UA Nodeset2.xml files containing type definitions

Figure 10 provides an overview of the connections between the files and which tools consume them. The Download File Creation Tool consumes the .opas import/export file to generate a download file as defined in Chapter 6. The HMI & Alarm Tool represents a tool that does not modify the application whose configuration is shown in Figure 10, but rather creates a configuration for other components that builds on this application. An example would be an alarm configuration as described in O-PAS Part 6.3 (Information and Exchange Models: Alarm and Events Configuration) that is added on top of an existing application. In order to be able to configure alarms, detailed knowledge of the existing application is needed.

Not shown in Figure 10 but also involved is a tool that is used to create the application. Such a tool would be responsible for creating the Nodeset2.xml and code file, as well as adding the application-relevant parts to the AML and the connections between them. This tool would be responsible for maintaining consistency between the three files.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

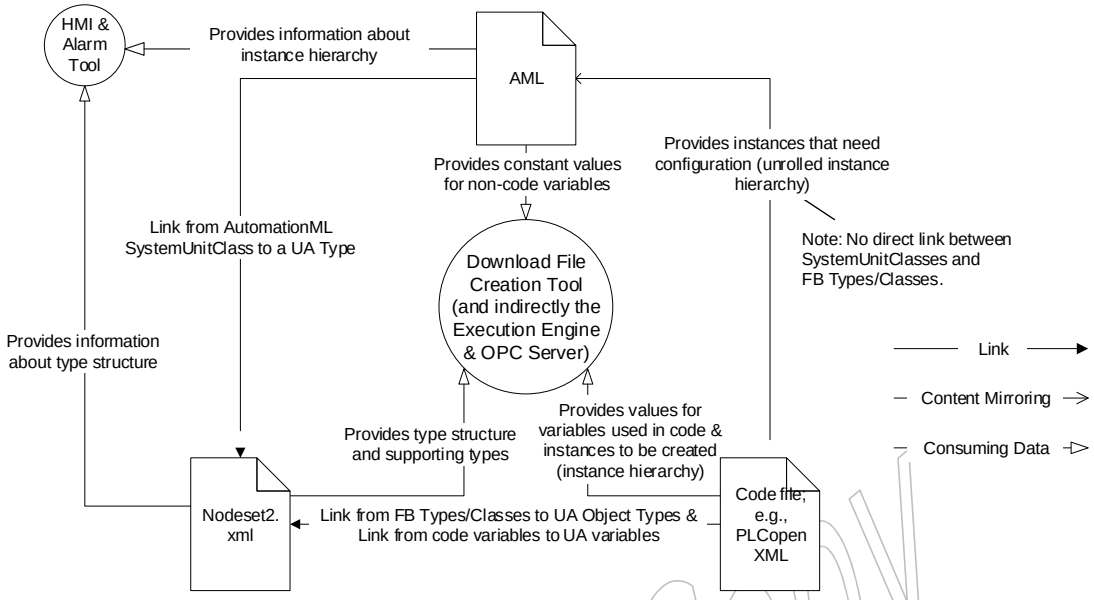


Figure 10: Common Relationship between AML, Nodeset XML, and Code Files

The content type for the Nodeset2.xml OPC UA nodeset files is "application/vnd.opc.ua.nodeset2+xml". The content type for the code file depends on the respective code file.

Relationships in a package from the AML or code file to the Nodeset2.xml that exist to provide UA type information use the relationship type http://opengroup.org/open-process-automation/O-PAS/relationships/ua-type-info.

F.2 Interaction between the Files

The Nodeset2.xml contains:

- UA object type definitions for function blocks/classes defined in the code:
  - They can contain sub-objects such as folders or functional groups to organize variables
  - They can contain variables for which the values are dynamically provided by the code
  - They can contain variables that are irrelevant for the code but whose value is configurable on a per-instance level (e.g., value precision)
  - They can contain variables that have constant values and are the same across all instances (e.g., descriptions)
- Supporting UA types needed by the object types above
- Localization for localized texts

The code file contains:

- The primary InstanceHierarchy, commonly created mostly by instantiating the top-level types; it contains only the instantiation of the top-level type – all contained instances must be built using type information

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

- Code for the function block/classes (types)
- Default values (potentially instance-specific) for all code variables

The AML contains:

- A simplified copy of the InstanceHierarchy that is completely expanded; i.e., all instances of all function blocks/classes are explicitly present (each instance has its own element in the AML)
- Configured values for instance-specific variables that are in UA but not the code

The links between the three files can be broken into two categories: type-level linkages and instance-level linkages.

F.2.1 Type Linkage

In the code file every type (function block or class type) contains a link to a UA object type. Variables in the function block/class type can also be linked to UA variables within the object type. Unlinked code variables are not shown in the OPC UA server. All information about the links is stored in the code file; no information about the link is part of the Nodeset2.xml.

In the AML file, an SUC is created for each UA object type. This SUC contains a link to the UA object type. The SUC contains elements for all UA variables that are not linked to code but should still be instance-level configurable. See Section E.1.10 for details of those elements. Again, all link information is in the AML; no information about the link is part of the Nodeset2.xml.

On a type level there is no link between the AML and the function block/class type in the code file; however, a general link to the file can be provided to decrease the search effort required by tools.

Links to UA object types are recommended to use NodeId and a filename; links to UA variables in an object type should use a BrowsePath within the UA object type.

F.2.2 Instance Linkage

For the purpose of this document, only OPC UA type definitions in the Nodeset2.xml file are relevant. There is no instance-level linking between the Nodeset file and AML or code files, only type-level linking.

Code files usually create an InstanceHierarchy by instantiating the top-level type; commonly individual instances are not explicitly shown. The existence of individual instances is implied using the root instance and type information. To allow configuration of individual instances the AML file contains the complete, expanded InstanceHierarchy. This means that there is an AML element for each individual instance and instance variable. Each variable can be immediately seen anywhere in the InstanceHierarchy without looking at any type information.

F.3 Consuming the Package

In general, there are two types of tools that will operate on a part of a package containing code: those that understand the code and those that don’t. Usually the only tools that understand the

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

code are tools used to edit the code and to create a download file from the code. Often both of these functionalities are provided by the same tool.

The second type of tools are those that don't understand the code. Often these tools provide additional functionality that does not influence the execution of the application, but rather adds to it. Common examples are tools to configure alarming (see O-PAS Part 6.3 – Information and Exchange Models: Alarm and Events Configuration), HMIs, or historians.

F.3.1 Download File Creation Tools

In order to create a download file that provides the execution engine and its OPC UA server with all the information it needs, a Download File Creation Tool needs information from all three files: the AML, the UA Nodeset2.xml, and the code file.

The primary file for this tool is the code file. It provides all the information about the application logic to be executed and the hierarchy of instances within the application. Based on this file the UA objects are created within the UA server.

The UA object types to be used for the instances are supplemented by the Nodeset2.xml file. Additionally, the file contains other UA types needed within the object such as special UA variable types.

The AML file adds values for UA variables that are not part of the code but should still be configurable for individual instances/UA objects.

In general, the values for UA variables can have three sources (in order of precedence): they can come from variables within the code, be supplied a constant value in the AML, or use the value defined in the UA object type definition.

F.3.2 HMI or Alarm Tools

HMI or alarm tools are examples of tools that are not directly involved in creating application logic. They don't know anything about specific programming languages or standards like IEC 61131 or IEC 61499 and operate on the UA Information Model of one or more other applications. Generally, those tools are not able to process the code files and use only the AML and Nodeset2.xml file. The AML provides them with information about the InstanceHierarchy – i.e., which instances exist – while the nodeset file provides the details of the different objects in the InstanceHierarchy and which variables they contain. Together, the two files provide a tool with a full understanding of how the UA Information Model for any application will look at runtime.

F.4 AML Definitions

The necessary AML attribute types and interface classes to create the links from the AML file to the Nodeset2.xml file are defined in Section 5.2.4.

F.5 Example

The following example shows how the mechanism described in this section can be applied. The example uses a function block defined in a pseudo-language and the link information is stored in a format typically used for documentation comments.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

The function block used in the example is a Counter with a variable for the Increment. Every time the block is executed the Counter goes up by Increment. Those two variables are part of the UA Information Model and the values for the UA variables are supplied by variables in the code.

In addition, the UA Information Model includes additional variables that are not used by the code. The HMIOrder variable should be configurable for individual instances, so each Counter has a different value so they are ordered properly on the HMI. The HMIOrder is completely irrelevant for the execution of the code. The definition variables are examples for nodes that are part of the UA Information Model but neither connected to code nor do they need to be configurable for individual instances.

F.5.1 Example Code

```
///<UAObjectType>
///<NodeId>s=CounterTypeNodeId</NodeId>
///<NamespaceURI> baytown.xom.com/blocks</NamespaceURI>
///</UAObjectType>
class Counter_FB
{
    ///<UAVariable>
    ///<BrowsePathSegment path="Increment" namespace="baytown.xom.com/blocks">
    ///</UAVariable>
    Int Inc;

    ///<UAVariable>
    ///<BrowsePathSegment path="Count" namespace="baytown.xom.com/blocks">
    ///</UAVariable>
    Int Count;

    // Constructor where initial values for code variables are set
    Counter_FB(Int _Inc, Int _Count)
    {
        Inc = _Inc;
        Count = _Count;
    }

    void main()
    {
        // internal variables not in UA
        Int i = 1;
        Float f = 1.23;
        ...
        Count += Inc;
    }
}

// instance declarations
static Counter_FB counterA = new Counter_FB(7,0);
static Counter_FB counterB = new Counter_FB(-100,-1);

void main()
{
    counterA.main();
    counterB.main();
}
```

F.5.2 Example UA Type Definition

The UA NamespaceURI for the function block in this example is baytown.xom.com/blocks.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

The Definition variables are from the UA DataItemType and thus use the UA standard NamespaceURI.

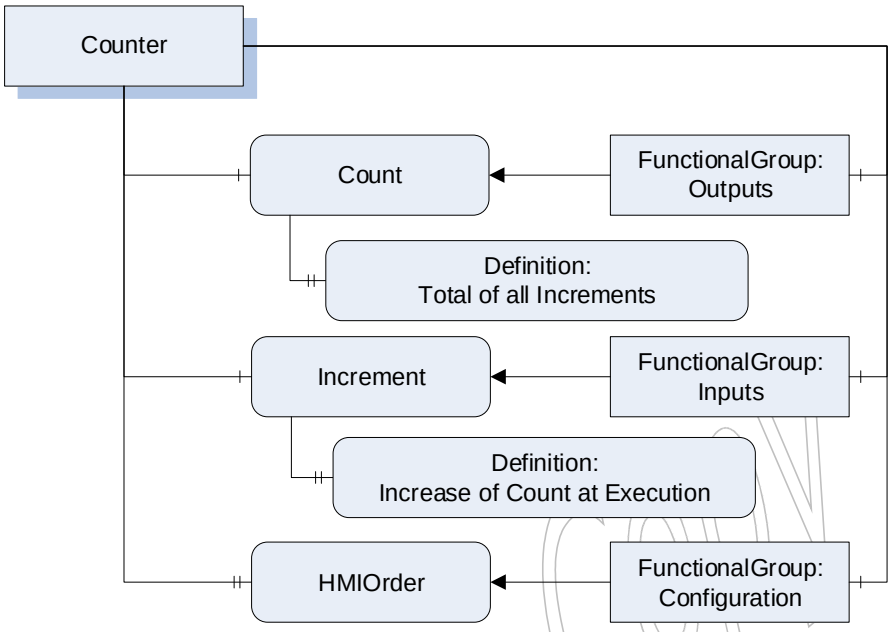


Figure 11: Example UA Object Type for a Counter Function Block

F.5.3 AML SUC

Figure 12 shows the overview of the AML SUC for the Counter function block and Table 25 provides the values of the attributes for the elements that make up the SUC.

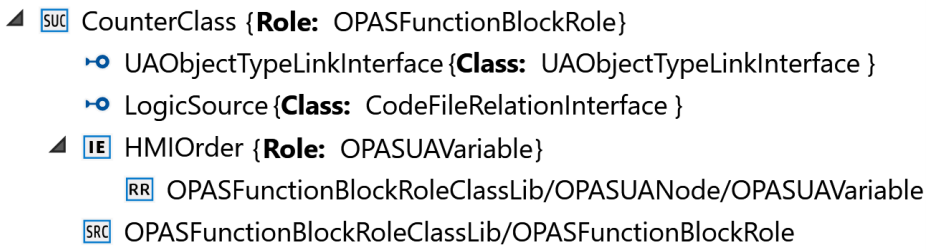


Figure 12: Example AML SUC for a Counter Function Block

Table 25: Attributes for AML SUC for a Counter Function Block

Element	Attribute	Value	Comment
UAObjectTypeLinkInterface	RelId	rId3	“rId3” references an Open Packaging Conventions relationship for the AML part in the package. It points to an OPC UA Nodeset2.xml file containing the object type.
	NodeId/Namespace URI	baytown.xom.com/blocks	This is the namespace used for user-defined function blocks in this application.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Element	Attribute	Value	Comment
	NodeId/StringNodeId	CounterTypeNodeId	This is the NodeId for the object type. Note that the other three attributes to specify a NodeId (Numeric, GUID, Opaque) are not present.
LogicSource	RelId	rId18	“rId18” references an Open Packaging Conventions relationship for the AML part in the package. It points to a code file with the contents described in Section F.3.2.
HMIOrder	BrowsepathSegmentURI	baytown.xom.com/blocks	URI complementing “HMIOrder” to build the BorwsePath pointing to the HMIOrder UA variable inside the Counter object type.
	Value		This attribute will contain the value for each instance of the function block. It is not set for the SUC definition.

F.5.4 AML InstanceHierarchy

The InstanceHierarchy in the AML file corresponding to the instances defined in the code in this example would look as follows:

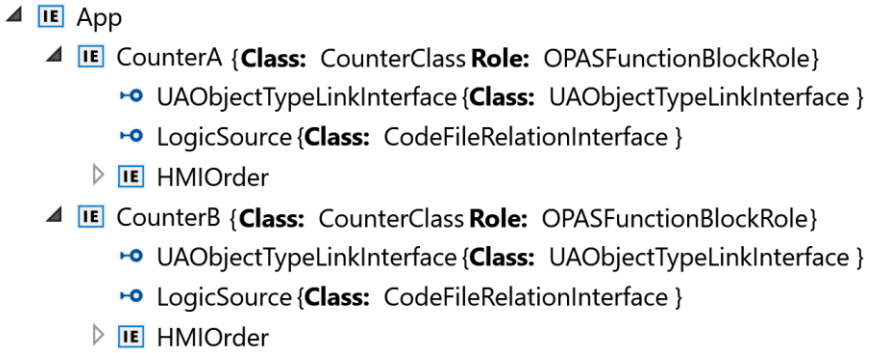


Figure 13: AML InstanceHierarchy for Counter Example

The names of the elements CounterA and CounterB refer to the BrowseNames of the UA objects that are created in the UA server of the execution engine for the two instances of the function block being created in the code file.

The instance-specific attributes set in this InstanceHierarchy are the value of HMIOrder, which is set to 1 for CounterB and 2 for CounterA, and the description for CounterA and CounterB. All other attributes maintain the value they have on the SUC.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

F.5.5 OPC UA Information Model in the Execution Engine

When this example is compiled and downloaded to an execution engine, the UA Information Model shown in Figure 14 would be shown in the UA server for the execution engine. Note that it is *not* stored in the .opas file at any time.

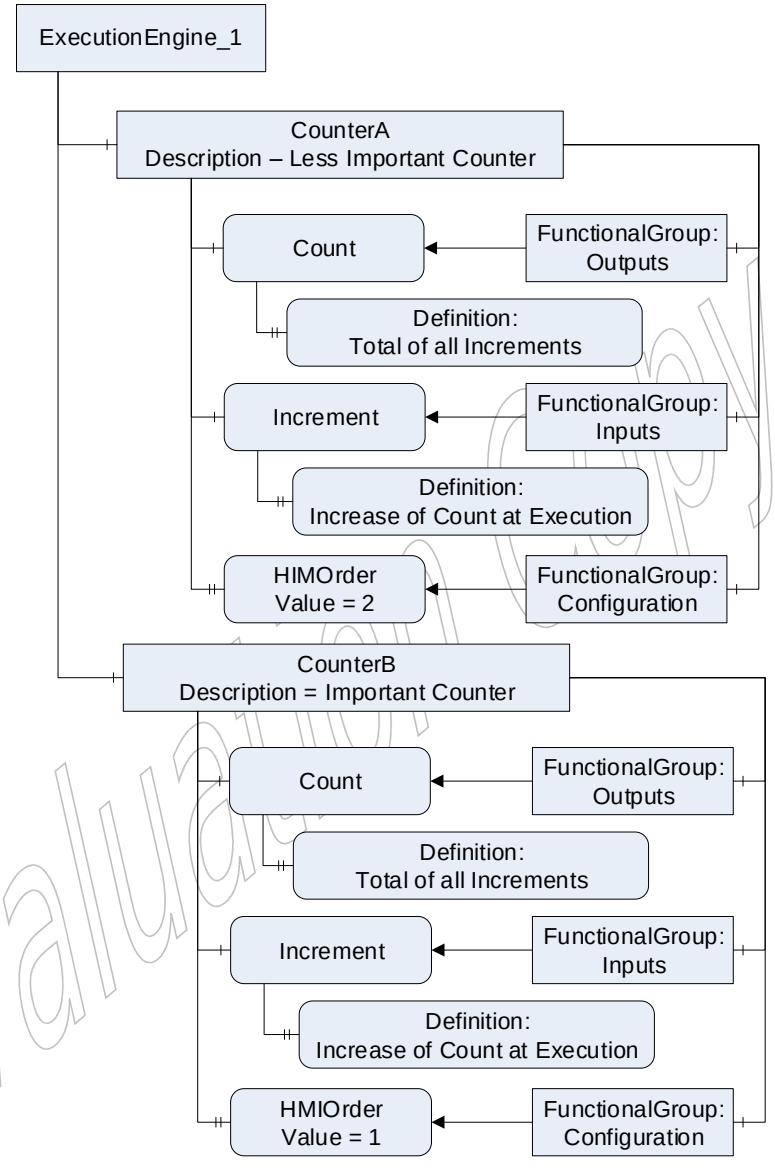


Figure 14: OPC UA Information Model for an Execution Engine Running the Counter Example

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

G Formal AML Definitions (Normative)

Formal, normative AML definitions can be found in the opas_part_6_2.aml file at: http://open-process-automation.projects.opengroup.org/twg/part-6-information-models/part-6.2-basic-configuration/DOCX/index.html.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

H Abbreviations

For the OPAF Glossary and Abbreviations, see the definitions at: www.opengroup.org/open-process-automation-forum/glossary.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Index

AML 4, 10, 44
Application Management Interface 30
application management states 30
CAEX 10
conformance 2
connectable parameters 21
DCN 5
download/upload 27, 28
execution engine..... 62, 65
facets 40
import/export format 1, 10
information model 8
interface class 16
Layer-F 1
Layer-F applications..... 30
localized text 21
NAMUR MTP..... 5
OBC-F-101..... 40
O-PAS Part-Properties 41
OPC UA 46
Open Packaging Conventions 10, 22
package 11
package file 10
primary environment..... 28
profiles 40
role class..... 17
secondary environment 28
StateMachine..... 30
SUC..... 14
UA BrowsePath..... 48
UA Information Model 48, 65
UA-AliasName..... 48
UA-BrowsePath 49
UA-Description 50
UA-FindAliasType..... 48
UA-NodeIdType 48
UML..... 1, 43
UML Class Model..... 44
XSD schema..... 41

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution